



**COMSATS University Islamabad,
Sahiwal Campus, Pakistan**

DiagnoXpert

By

Eman Khadim

CIIT/FA22-BCS-066/SWL

Saira Bano

CIIT/ FA22-BCS-107/SWL

Wasib Mehmood

CIIT/ FA22-BCS-130/SWL

Supervisor

Dr. Yawar Abbas Abid

Bachelor of Science in Computer Science (2022-2026)



**COMSATS University Islamabad,
Sahiwal Campus, Pakistan**

DiagnoXpert

A Project Presented To

**COMSATS University Islamabad,
Sahiwal Campus**

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Eman Khadim

CIIT/FA22-BCS-066/SWL

Saira Bano

CIIT/FA22-BCS-107/SWL

Wasib Mehmood

CIIT/FA22-BCS-130/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied the report entirely based on our efforts. If any part of this project is proved to be copied from any source or found to be the reproduction of some other. We will stand by the consequences. No portion of the work presented has been submitted because of any application for any other degree or qualification of this or any other university or institute of learning.

Eman Khadim

Saira Bano

Wasib Mehmood

CERTIFICATE OF APPROVAL

This is to certify that the final year project of BS(CS) “**DiagnoXpert**” was developed by Eman Khadim (CIIT/FA22-BCS-066/SWL), Saira Bano (CIIT/FA22-BCS-107/SWL) and Wasib Mehmood (CIIT/FA22-BCS-130/SWL) under the supervision of “**Dr. Yawar Abbas Abid**” and that in his opinion; it is fully adequate, in scope and quality for the degree of bachelors of science in Computer Science.

Supervisor

Dr. Yawar Abbas Abid

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr Syed M Awais

Assistant Professor

Department of Computer Science Air University Multan

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

DiagnoXpert is an AI powered medical report analysis software that turns confusing medical reports into clear and comforting health insights. In today's healthcare system, patients often feel scared, helpless, and worried when they receive blood test reports, lipid profiles, liver function tests, and other reports full of hard medical words and numbers they don't understand. Using smart OCR, powerful AI language models, and machine learning, DiagnoXpert quickly reads, understands, and explains the report in simple words. It reduces the patient's fear, shows what really matters, and gives the patient clarity and confidence to take charge of their health.

The platform offers many helpful **patient centered features**, including risk analysis, abnormality detection with severity levels, easy to understand medical explanations, personalized health scores, and tailored lifestyle and nutritional recommendations. Patients can upload their diagnostic reports in multiple formats including text based PDFs and scanned images through an intuitive web interface. The system processes these uploads using Tesseract.js OCR and pdf-parse for text extraction, followed by biomarker parsing and AI driven analysis via the Llama3-OpenBioLLM model hosted on the Hugging Face API.

A key distinguishing feature of DiagnoXpert is its integrated **Doctor Consultation Module**, which enables patients to connect directly with verified healthcare professionals for real-time consultation. Doctors can review AI generated analysis, add clinical notes, prescribe medications, and suggest further investigations all within the platform. This seamless integration of AI assistance and human expertise creates a holistic, patient-first healthcare support system.

The platform further enhances accessibility through a **chatbot** that allows users to ask health related questions using natural language both typed and spoken and receive context aware, patient friendly responses. A reporting history module enables longitudinal health trend tracking, while a rating and review system supports continuous improvement of AI models through iterative user feedback.

DiagnoXpert is built on a modern, scalable technology stack comprising Next.js 16 with React 19, TypeScript, MongoDB with Mongoose, Redux Toolkit for state management, and Cloudinary for cloud media storage. The architecture follows modular, component based design principles, ensuring maintainability and ease of future expansion including planned support for radiology report analysis and integration with hospital Electronic Medical Records systems.

Ultimately, **DiagnoXpert** represents a transformative step toward democratizing healthcare information. By empowering patients with the knowledge and tools to understand their own health data, the platform reduces dependence on overburdened clinical staff for basic report interpretation, promotes early detection of health risks, and fosters more informed, confident participation in personal healthcare decisions.

Acknowledgement

All praise is to the almighty Allah, who bestowed upon us a minute portion of his boundless knowledge by which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “**Dr. Yawar Abbas Abid**” whose expert guidance, insightful feedback, and consistent encouragement were invaluable at every stage of this project. Without his supervision, advice, and valuable guidance, the completion of this project would have been doubtful.

We are also thankful to our parents and family, who have been a constant source of encouragement for us and have brought us the values of honesty & hard work.

We would also like to thank our friends for their constant encouragement and support. They motivated us when we felt tired or worried, cheered us up, and helped us stay positive throughout the project.

Eman Khadim

Saira Bano

Wasib Mehmood

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CBC	Complete Blood Count
EMR	Electronic Medical Record
GDPR	General Data Protection Regulation
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
ML	Machine Learning
MVC	Model View Controller
NFR	Non-Functional Requirement
NLP	Natural Language Processing
OCR	Optical Character Recognition
OOM	Object Oriented Methodology
OOP	Object Oriented Programming
RBAC	Role Based Access Control
REST	Representational State Transfer

Table of Contents

1	Introduction	1
1.1	Brief Overview	1
1.2	Relevance to Course Modules	3
1.3	Project Background	4
1.4	Literature Review	6
1.5	Analysis from Literature Review	8
2	Problem Definition	13
2.1	Problem Statement	13
2.2	Deliverable and Development Requirements.....	15
2.3	Methodology and Software Lifecycle for This Project.....	19
1.6.1	Agile Methodology	19
1.6.2	Iterative Process Model.....	20
3	Requirement Analysis.....	20
3.1	Use Case Descriptions.....	23
3.2	Functional Requirements.....	31
3.3	Non-functional Requirements	34
4	Design and Architecture	33
4.1	System Architecture	33
4.2	Process Flow/Representation	36
4.3	Sequence Diagram.....	39
4.4	Class Diagram	41
4.5	Activity Diagram.....	45
4.6	State Diagram	47
4.7	Design Flow Diagram	49
5	Implementation	50
5.1	Setup Development Environment:	50
5.2	External APIs	55
5.3	User interface	56
6	Testing	67
7	Conclusion and Future Work	71
7.1	Conclusion.....	71

7.2	Future Work	72
References		74

List of Figures

Figure 2.1 Agile Model Diagram.....	19
Figure 3.1 DiagnoXpert Use Case Diagram	22
Figure 4.1 DiagnoXpert System Architecture	34
Figure 4.2 DiagnoXpert Process Flow.....	38
Figure 4.3 DiagnoXpert Sequence Diagram.....	41
Figure 4.4 DiagnoXpert Class Diagram.....	44
Figure 4.5 DiagnoXpert Activity Diagram	47
Figure 4.6 DiagnoXpert State Diagram	49
Figure 4.8 DiagnoXpert Design Flow Diagram.....	51
Figure 5.1 DiagnoXpert Login/Signup Page	52
Figure 5.2 DiagnoXpert Landing Page	57
Figure 5.3 DiagnoXpert Feature Section	57
Figure 5.4 DiagnoXpert Patient Profile	58
Figure 5.5 DiagnoXpert LLM Interface.....	59
Figure 5.6 DiagnoXpert LLM Report Analysis Results	60
Figure 5.7 DiagnoXpert Doctor Module.....	60
Figure 5.8 DiagnoXpert Doctor Listing and Filtering Screen	61
Figure 5.9 DiagnoXpert Doctor profile	62
Figure 5.10 DiagnoXpert Patient-Doctor Screen.....	62
Figure 5.11 DiagnoXpert AI Chatbot Interface	63
Figure 5.12 DiagnoXpert Admin Login Interface	64
Figure 5.13 DiagnoXpert Admin OTP Confirmation	64
Figure 5.14 DiagnoXpert Admin Dashboard Overview	65
Figure 5.15 DiagnoXpert Doctor Profile Review and Field-Level Approval	65

List of Tables

Table 3.1 Register/Login	23
Table 3.2 Approve Doctor	23
Table 3.3 Reject Doctor	24
Table 3.4 Upload Report	26
Table 3.5 View Extracted Data	26
Table 3.6 View Analysis	27
Table 3.7 Doctor Consultation.....	28
Table 3.8 Chatbot Interaction.....	29
Table 3.9 View Report History	30
Table 3.10 Manage Users	31
Table 5.1 External APIs.....	55
Table 6.2 Unit Testing 1: Patient Login.....	67
Table 6.3 Functional Testing	69
Table 6.4 Integration Testing.....	72
Table 6.5 Automated Testing.....	73

Chapter 1

Introduction

1 Introduction

Understanding medical reports is one of the biggest challenges faced by patients today. Diagnostic reports such as blood tests, lipid profiles, liver function tests, and other medical investigations are filled with technical terminology and numerical values that are difficult for ordinary people to interpret. This knowledge gap often creates fear, confusion, and unnecessary anxiety among patients.

In Pakistan and many other developing countries, patients frequently have limited and delayed access to doctors for report explanations. As a result, they either remain worried about their health condition or, in some cases, ignore important warning signs altogether. DiagnoXpert is developed to address this critical problem. It is an AI based platform that allows patients to upload their medical reports easily through a web interface. The system automatically extracts data using Optical Character Recognition, analyzes it using a Large Language Model, and presents clear, simple explanations in patient friendly language. It highlights abnormalities, explains their potential impact on health, and provides practical lifestyle and nutritional recommendations. To ensure patients can trust the doctors they consult through the platform, DiagnoXpert also includes an admin verification system, where doctor registrations are reviewed and approved before they are allowed to interact with patients, adding a layer of credibility and safety to the consultation process.

This project combines Artificial Intelligence, Machine Learning, OCR technology, and full-stack web development to make healthcare information more accessible and understandable for patients. The primary aim of **DiagnoXpert** is to empower patients with better understanding and confidence so they can take proper care of their health and communicate more effectively with their doctors when needed.

1.1 Brief Overview

DiagnoXpert is a web based platform designed to solve a major real life problem faced by patients across Pakistan and many other countries. The primary issue being addressed is that medical reports are filled with complex medical terms, abbreviations, and numerical values that are extremely difficult for ordinary people to understand. This often leaves patients feeling anxious, confused, and powerless about their own health. In many areas, particularly rural regions of Pakistan, access to doctors is limited, and patients may have to wait days or even weeks to receive

a proper explanation of their reports. During this waiting period, many people either worry unnecessarily or overlook potentially serious health issues.

To address this challenge, **DiagnoXpert** provides an easy, intelligent, and secure solution. The platform features role-based access for both patients and doctors. Users can register or log in to the website and create personalized profiles containing basic details such as name, age, gender, and medical history. The system automatically assigns appropriate roles, ensuring secure and relevant access for each type of user.

Once logged in, patients can simply upload their medical reports in supported formats. Using advanced OCR technology powered by Tesseract.js and PDF text extraction via pdf-parse, the system automatically extracts key information from the report. This extracted data is then analyzed using a Large Language Model, delivering a clear and simple summary in everyday language. The analysis highlights normal and abnormal values, explains their potential impact on the patient's health, provides a personalized health score, and offers practical diet and lifestyle recommendations. Patients can further interact with an AI powered chatbot to ask questions about their report and receive instant, context-aware answers.

The platform also enables patients to securely contact their doctor directly through the website. They can share reports, send messages, and request consultations. Doctors have a dedicated dashboard to review shared reports, view AI generated summaries, add clinical notes, and communicate back with patients efficiently. DiagnoXpert includes a reliable notification system that sends timely email alerts to users via Nodemailer.

All uploaded reports and sensitive user data are protected through a secure file storage system backed by Cloudinary for cloud media management, combined with strong security measures including password hashing via bcryptjs, secure session based authentication through NextAuth.js, and role-based access control ensuring complete privacy and safety of health information.

The user interface is built around intuitive design principles, prioritizing ease of navigation and accessibility. The clean, simple, and user-friendly design built with Tailwind CSS and shadcn/ui ensures that both patients and doctors can use the platform comfortably without any technical difficulty. The project seamlessly combines Artificial Intelligence, Machine Learning, Large Language Models, full stack web development using Next.js 16 with React 19, and secure cloud technologies.

DiagnoXpert does not aim to replace doctors. Instead, it empowers patients with clear knowledge and confidence so they can better understand their own health and engage in more meaningful conversations with their healthcare providers. By making healthcare more accessible, understandable, and secure, DiagnoXpert contributes toward improved health awareness and better healthcare experiences for everyone in Pakistan.

1.2 Relevance to Course Modules

The development of DiagnoXpert draws extensively on the interdisciplinary knowledge acquired throughout the Bachelor of Science in Computer Science program at COMSATS University Islamabad, Sahiwal Campus. The project serves as a synthesis of theoretical foundations and practical skills developed across multiple courses, demonstrating how a modern software system integrates concepts from diverse domains of computer science.

Artificial Intelligence and Machine Learning: The core of DiagnoXpert's functionality rests on AI and ML principles studied throughout the program. Machine learning concepts underpin the disease prediction, risk scoring, and abnormality detection modules. The application of a pretrained biomedical language model specifically Llama3-OpenBioLLM hosted via the Hugging Face API reflects practical application of concepts in Natural Language Processing and transfer learning. The biomarker parsing and risk scoring algorithms directly apply supervised learning and rule-based classification techniques covered in AI and ML coursework.

Database Management Systems: MongoDB with Mongoose ODM is employed as the primary data store for user profiles, uploaded reports, analysis results, and consultation records. Course knowledge in database design, schema modeling, indexing, query optimization, and data security directly informed the data architecture of DiagnoXpert. The use of a NoSQL document store reflects an understanding of when non-relational data models outperform traditional relational schemas for semi-structured and variable medical data.

Software Engineering and Object-Oriented Programming: DiagnoXpert follows object-oriented methodology and Agile development principles learned in Software Engineering and OOP courses. The system's modular architecture with independent components for OCR extraction, LLM analysis, health scoring, consultation management, and notification services directly applies principles of encapsulation, abstraction, and separation of concerns. Agile sprints,

iterative development cycles, and continuous testing practices were adopted throughout the project lifecycle.

Web Application Development: The full-stack interface was developed using Next.js 16 with React 19 and TypeScript, applying skills from web development coursework. Responsive design with Tailwind CSS, global state management via Redux Toolkit, API integration, and component based architecture was all central to building an accessible and performant user experience. Next.js API routes serve as the backend layer, reflecting an understanding of full-stack architecture, server side rendering, and HTTP communication protocols.

Data Science and Analytics: Statistical analysis underpins the health trend tracking and biomarker comparison modules. Data science concepts including descriptive statistics, deviation analysis, and data visualization were applied to generate meaningful health scores and graphical representations of patient data over time, enabling longitudinal monitoring of patient health.

Information Security and Ethics: Patient data is among the most sensitive categories of personal information. DiagnoXpert incorporates security principles studied in information security courses, including password hashing via bcryptjs, secure session-based authentication through NextAuth.js, and role based access control enforced at both the frontend and API levels. Adherence to healthcare data privacy standards reflects the ethical dimensions of computing discussed throughout the program.

The interdisciplinary nature of the Computer Science degree enabled the team to approach DiagnoXpert holistically, balancing technical rigor with user centric design and health awareness. By integrating programming expertise, software engineering best practices, and communication skills, we transformed DiagnoXpert from a conceptual solution into a functional application with the potential to meaningfully improve healthcare accessibility. This project not only demonstrates our technical proficiency but also reflects our ability to synthesize academic knowledge into a real-world innovation that addresses a genuine societal need.

1.3 Project Background

A major gap in patient friendly healthcare information led to the creation of DiagnoXpert. In Pakistan and many developing countries, people regularly undergo blood tests, urine tests, lipid profiles, and other laboratory investigations. They receive printed or digital reports filled with

difficult medical terminology and raw numerical values that are largely inaccessible to the average person.

Without consultation from a doctor who is often not easily available, particularly in rural areas, villages, and small towns, patients have no reliable way to understand what their results truly mean. **For example**, if a report shows high uric acid, low hemoglobin, or abnormal liver enzymes such as ALT and AST, most patients cannot determine how serious the condition is or what steps they should take next.

Existing platforms such as **Ada Health, SiPhox Health, and Buoy Health** offer useful functionality for symptom checking and general health tracking. Within Pakistan, **Marham.pk** is widely used to consult doctors online, obtain medicine prescriptions, and book laboratory tests. However, even after booking a lab test through Marham.pk, patients must still visit the laboratory physically for sample collection. Once the test is completed, they receive the report online, but the platform provides no meaningful explanation of what the numbers and medical terms actually indicate for their personal health leaving them still dependent on a follow up doctor consultation.

Most other available tools do not accept raw medical reports directly. Instead, they require users to manually enter structured data field by field. Their interfaces are typically designed for medical professionals rather than ordinary patients. No complete solution existed that could accept raw diagnostic reports in any format, interpret them using AI, explain the results in plain language, and simultaneously allow patients to connect with verified doctors all within a single platform.

That is why DiagnoXpert was created. It uses Tesseract.js OCR technology to extract text from scanned and digital reports, the Llama3-OpenBioLLM language model hosted via the Hugging Face API to understand and explain the results, and Cloudinary backed cloud storage to make file management fast and reliable. The platform is powerful enough to deliver meaningful health insights, yet simple enough for any patient including those with no medical or technical background to use comfortably.

DiagnoXpert is not intended to replace the physician. Rather, it is designed to augment the patient's understanding and preparation for clinical interactions, thereby improving health literacy and promoting more informed, confident engagement with the healthcare system.

1.4 Literature Review

A thorough review of existing academic literature and commercially deployed systems was conducted to situate DiagnoXpert within the current state of ai assisted healthcare and to inform the design decisions made throughout the project.

AI in Clinical Decision Support

The application of artificial intelligence to clinical decision support has been a subject of active research for over two decades. Friedman and Hripcsak (1999) (Hripcsak, 1999) provided one of the earliest comprehensive assessments of natural language processing in medicine, identifying the extraction of structured clinical information from free text as a foundational challenge that NLP was uniquely positioned to address. Their work anticipated the trajectory of biomedical language processing, which has culminated in the development of domain specific large language models capable of contextual medical reasoning.

Their survey documented the growing viability of machine learning models for disease prediction from structured laboratory data, particularly in the context of metabolic disorders, cardiovascular risk, and haematological abnormalities. Their findings directly informed the design of DiagnoXpert's disease prediction and risk scoring modules.

Machine Learning in Medical Data Analysis

Kazakov, Averchenkov, and Kulagina (O. D. Kazakov, 2019) explored machine learning methods applied to medical data analysis, with particular attention to classification algorithms for laboratory diagnostics. Their work demonstrated the effectiveness of ensemble methods and neural networks in identifying pathological patterns in haematological data a methodology reflected in DiagnoXpert's biomarker classification pipeline.

Guncár et al. (G. Gunčar, 2018) presented a landmark application of machine learning to haematological diagnosis, achieving high accuracy in distinguishing between anaemia subtypes, thrombocytopenic conditions, and infectious processes from complete blood count data. This study validated the clinical utility of AI driven blood test analysis and provided an empirical foundation for DiagnoXpert's focus on CBC analysis as a primary diagnostic modality.

Related Commercial Systems

Ada Health operates as a symptom checking platform with limited integration of laboratory data. While user-friendly and multilingual, it is primarily symptom driven and does not offer OCR based report analysis or longitudinal biomarker tracking. Its reliance on user reported symptoms rather than objective diagnostic data limits its analytical depth.

SiPhox Health's Sai platform focuses specifically on blood test analysis, offering biomarker extraction, trend tracking, and lifestyle recommendations. However, it is limited to blood tests and does not support a voice interface or direct consultation with healthcare professionals. Its subscription based model further restricts accessibility for general users.

Buoy Health provides AI driven triage recommendations based on symptom inputs, with basic integration of laboratory data. It is effective for initial triage but offers limited personalization and does not perform deep diagnostic report analysis.

Marham.pk functions primarily as a telemedicine marketplace and doctor appointment booking platform, supporting online consultations and limited lab test booking. While it offers convenient access to verified doctors across Pakistan and supports video consultations, it lacks OCR based medical report analysis, automated biomarker tracking, or AI driven personalized health insights. Its focus remains on facilitating connections between patients and healthcare providers rather than providing data driven report interpretation or longitudinal health monitoring.

OCR in Healthcare

The application of optical character recognition to medical document processing has been facilitated by the adoption of Tesseract.js, an open source OCR engine capable of extracting text from structured and semi structured documents within a JavaScript runtime environment. Research in this area has highlighted both the potential and the limitations of OCR in clinical contexts: accuracy is high for clearly printed, standardized laboratory report formats, but degrades significantly for handwritten notes, low resolution scans, or non-standard report layouts. DiagnoXpert's scope is accordingly constrained to printed and digitally generated diagnostic reports.

Patient Health Literacy

Research consistently demonstrates that low health literacy is associated with poorer health outcomes, higher rates of hospitalization, and reduced adherence to treatment regimens. Interventions that improve patients' understanding of their diagnostic data have been shown to increase engagement with preventive care and reduce the burden on healthcare systems. DiagnoXpert is designed with this evidence base in mind, prioritizing plain language explanations, visual abnormality highlighting, and severity classification to maximize the comprehension and actionability of diagnostic information for patients across all literacy levels.

1.5 Analysis from Literature Review

The literature review yields several key insights that directly shaped the design and scope of DiagnoXpert.

First, AI powered analysis of laboratory data is not merely feasible but clinically validated multiple peer reviewed studies demonstrate that machine learning models can achieve clinician level accuracy in identifying haematological abnormalities and predicting metabolic risk from structured blood test data. This provides strong empirical justification for DiagnoXpert's core analytical functionality.

Second, the review reveals a persistent gap between the analytical capabilities of existing systems and the needs of non-clinical users. **Ada Health** and **Buoy Health** serve patients well in the symptom checking domain but do not process raw diagnostic documents. **SiPhox Health's Sai** platform approaches **DiagnoXpert's** domain more closely but remains limited to blood tests and lacks interactive doctor consultation. No existing system comprehensively addresses the patient's need to understand and act upon their complete diagnostic report within a single integrated platform.

Third, diagnostic reports in Pakistan and across South Asia are issued in a wide variety of formats and layouts. Many laboratories still produce reports as scanned PDFs that standard text processing software cannot read directly. DiagnoXpert addresses this challenge by supporting multiple document formats through a dual extraction pipeline using pdf parse and pdfjs-dist for digitally generated, text based PDFs, and Tesseract.js OCR for scanned image based PDFs allowing patients to upload any type of diagnostic report without additional preparation.

Fourth, the literature on health literacy reinforces the centrality of clear and accessible communication in any patient facing health platform. DiagnoXpert's use of LLM generated plain

language explanations, severity classification, and contextual lifestyle and nutritional recommendations aligns with evidence based best practices for improving patient understanding and promoting active engagement with personal health data.

Finally, the analysis of commercial systems highlights the competitive differentiation provided by DiagnoXpert's integrated Doctor Consultation Module. The ability to transition seamlessly from AI-generated report analysis to direct communication with a verified healthcare professional all within a single platform addresses a critical continuity of care need that no existing consumer health platform currently satisfies.

Chapter 2

Problem Definition

2 Problem Definition

2.1 Problem Statement

In today's healthcare system, medical test reports are important for diagnosing and monitoring health conditions. However, most patients find it difficult to understand reports such as blood tests, liver function tests, lipid profiles, and urinalysis because they contain medical terms, abbreviations, and numerical values that require professional knowledge. This lack of understanding can lead to confusion, anxiety, delayed medical attention, and poor health decisions. The problem is even more serious in developing countries like Pakistan, where access to healthcare professionals may be limited. While some health applications provide symptom checking or basic biomarker analysis, they do not offer a complete solution for interpreting medical reports in a simple and user friendly way. **DiagnoXpert** addresses this issue by enabling users to upload their medical reports and receive clear, AI generated explanations, helping them better understand their health, track their medical history and consult with doctor, and make informed decisions through a single platform.

Challenges:

The following specific challenges were identified through stakeholder interviews, literature review, and detailed assessment of existing healthcare applications.

i. Complexity of Medical Terminology:

Most patients do not possess the medical background required to understand clinical terms, abbreviations, biomarker names, and reference ranges. A single blood report can contain dozens of such terms, leaving patients confused and anxious about whether their results are normal or potentially dangerous. Without proper explanation, they often misinterpret the data or overlook important warning indicators. DiagnoXpert addresses this through LLM generated plain language explanations powered by Llama3-OpenBioLLM, which translates clinical findings into accessible, patient-friendly language.

ii. Heterogeneity of Report Formats:

Diagnostic laboratories across Pakistan and South Asia use highly diverse report templates. Some laboratories provide clean, digitally generated PDFs, while others deliver scanned PDFs with varying layouts, fonts, tables, and designs. This variation makes it extremely difficult for automated systems to reliably extract and interpret information from every report format. DiagnoXpert addresses this through a dual extraction pipeline pdf parse and pdfjs dist for text

based PDFs, and Tesseract.js OCR with Sharp based image preprocessing for scanned documents.

iii. Limited Access to Professional Guidance:

In many parts of Pakistan, particularly rural and underserved areas, patients face long waiting times or have limited access to qualified physicians. Even when reports are received, timely professional explanation is often unavailable. This delay increases anxiety and may cause patients to either worry unnecessarily or overlook serious health conditions. DiagnoXpert's integrated Doctor Consultation Module allows patients to share reports and communicate directly with verified healthcare professionals within the platform, reducing this critical gap.

iv. Absence of Longitudinal Context:

A single medical report captures health status at only one point in time. Clinicians, however, typically make more informed decisions by observing trends and changes across multiple reports over weeks or months. Most current tools do not support this historical comparison, limiting the depth of insight available to both patients and doctors. DiagnoXpert's report history module enables longitudinal tracking of biomarker values, allowing meaningful health trend analysis over time.

v. Digital Health Literacy:

A significant proportion of patients particularly elderly individuals and those from lower education backgrounds have limited familiarity with digital platforms. They require extremely simple, intuitive, and accessible interfaces. Complicated designs or excessive technical language can discourage adoption, undermining the platform's core purpose of making healthcare more accessible. DiagnoXpert's interface, built with Tailwind CSS v4 and shadcn/ui, is designed around clarity and ease of navigation, ensuring comfortable use regardless of technical background.

vi. Data Privacy and Security:

Medical information is among the most sensitive categories of personal data. Patients expect complete confidentiality, yet many digital health tools struggle to balance strong security measures with user friendly access. Any weakness in data protection can result in serious privacy breaches, erosion of user trust, and legal complications. DiagnoXpert addresses this through a layered security architecture comprising bcryptjs password hashing, session based authentication via NextAuth.js, role based access control enforced at both the frontend and API levels, and secure cloud media storage through Cloudinary.

2.2 Deliverable and Development Requirements

The DiagnoXpert is designed to provide a seamless, patient friendly, and efficient platform for AI powered medical report analysis and doctor consultation. The project has been broken down into clearly defined deliverables and development requirements, ensuring each component is robust, scalable, and aligned with user needs.

Deliverables

- Fully functional web application with dedicated patient and doctor portals.
- OCR and PDF text extraction pipeline supporting both digitally generated and scanned PDF report formats.
- AI driven biomarker parsing, abnormality detection, and health risk scoring engine.
- LLM based medical insight generation using the Llama3-OpenBioLLM model via the Hugging Face API.
- Personalized health score dashboard with graphical visualization of biomarker results.
- Doctor Consultation Module with real-time messaging and secure report sharing.
- AI powered chatbot for interactive health query resolution.
- Report history module for longitudinal health trend analysis.
- Secure, privacy compliant data management infrastructure.

Development Requirements

The following tools and frameworks were carefully selected to ensure high performance, security, and ease of use across the platform.

- Full-Stack Web Development:** DiagnoXpert is built as a full-stack web application using Next.js 16 with Turbopack and React 19, written entirely in TypeScript. Next.js serves as both the frontend rendering framework and the backend API layer through its built-in API routes, eliminating the need for a separate server framework. Global state is managed using Redux Toolkit, and the UI is constructed with Tailwind CSS v4 and shadcn/ui component library for a clean, accessible, and responsive experience across all devices.
- OCR and Document Extraction Pipeline:** To handle real world laboratory reports effectively, the system supports both digitally generated and scanned PDF formats. Pdf parse and pdfjs-dist are used for extracting text from clean, text based PDF files, while Tesseract.js with Sharp for image preprocessing processes scanned or image based PDFs.

This dual pipeline approach ensures reliable text extraction regardless of the report format or quality.

- iii. **Hugging Face API Integration for LLM Based Medical Analysis:** Advanced biomedical language model capabilities are integrated through the @huggingface/inference SDK. The Llama3-OpenBioLLM model analyzes extracted report data, generates patient friendly summaries, detects abnormalities, calculates personalized health scores, and provides practical diet and lifestyle recommendations tailored to the individual patient.
- iv. **MongoDB Database with Mongoose ODM:** MongoDB is used as the primary NoSQL database for storing user profiles, medical reports, chat history, consultation records, and analysis results. Mongoose provides structured schema modeling and query management on top of MongoDB, offering high scalability and strong data integrity as the platform grows.
- v. **Authentication and Security:** Secure user authentication is implemented using NextAuth.js with bcryptjs for password hashing. Role based access control is enforced at both the frontend and API route levels, ensuring that patients and doctors access only the features and data relevant to their respective roles. Uploaded files and media are securely managed through Cloudinary with next-cloudinary integration.
- vi. **Cloud Deployment and CI/CD Pipeline:** The application is deployed to Vercel, providing a scalable and globally distributed hosting environment optimized for Next.js. The CI/CD pipeline is managed through GitHub Actions with two automated workflows a continuous integration workflow that builds and verifies the project on Node.js 20.x and 22.x for every pull request, and a continuous deployment workflow that automatically ships to Vercel production upon every merge to the main branch.

These deliverables and development requirements collectively ensure the successful delivery and continuous improvement of the DiagnoXpert platform, in direct support of its mission to make healthcare information more accessible, understandable, and secure for patients and doctors alike.

2.3 Current System

DiagnoXpert is designed to address the challenges faced by patients in understanding and acting upon their medical diagnostic reports. Many individuals struggle with complex medical terminology, limited access to timely physician explanations, and the absence of a single platform that combines AI powered report analysis with direct doctor consultation. DiagnoXpert provides

a comprehensive digital solution that enables patients to upload their diagnostic reports, receive AI generated health insights in plain language, and connect with verified healthcare professionals all through a single, secure, and user friendly web platform.

The system offers intelligent report analysis, personalized health scoring, biomarker abnormality detection, and longitudinal health trend tracking. By integrating these functionalities with a Doctor Consultation Module and an AI powered chatbot, DiagnoXpert enhances the healthcare experience for both patients and doctors, ensuring accessibility, clarity, and continuity of care.

i. User Centric Web Interface

At the core of DiagnoXpert is a responsive web application built with Next.js 16, React 19, and TypeScript, styled using Tailwind CSS v4 and the shadcn/ui component library. The interface is designed to be clean, intuitive, and accessible to users of all technical backgrounds including elderly patients and those from low education backgrounds. Smooth navigation, clear visual hierarchy, and plain language outputs ensure that patients can manage their health information without confusion or technical difficulty.

ii. Report Upload and Document Extraction

DiagnoXpert supports the upload of medical reports in multiple formats through a Multer based file handling pipeline. For digitally generated, text based PDFs, text extraction is performed using pdf parse and pdfjs-dist. For scanned or image-based PDFs, Tesseract.js OCR is used in combination with Sharp for image preprocessing. This dual extraction pipeline ensures reliable and accurate text extraction regardless of the report format, layout, or scan quality submitted by the patient.

iii. AI Powered Biomarker Analysis and Health Scoring

Once text is extracted from the uploaded report, DiagnoXpert's analysis engine identifies and parses individual biomarkers, compares them against clinical reference ranges, and classifies each result by severity normal, borderline, or abnormal. The Llama3-OpenBioLLM model, accessed via the @huggingface/inference SDK, then generates patient-friendly explanations of the findings, highlights health risks, computes a personalized health score, and provides tailored diet and lifestyle recommendations. This transforms a complex clinical document into a clear, actionable health summary.

iv. Doctor Consultation Module

A key differentiating feature of DiagnoXpert is its integrated Doctor Consultation Module. Patients can securely share their uploaded reports and AI generated analysis with verified doctors registered on the platform. Doctors have access to a dedicated dashboard where they can review reports, view AI-generated summaries, add clinical notes, prescribe medications, and communicate directly with patients. This seamless transition from automated analysis to professional consultation addresses a critical continuity of care need not satisfied by any existing consumer health platform.

v. AI Powered Chatbot

DiagnoXpert includes an AI powered chatbot that allows patients to ask health related questions in natural language both typed and spoken and receive context aware, patient friendly responses. The chatbot is designed to help patients better understand their report findings, clarify medical terms, and receive guidance on next steps, providing an additional layer of interactive support beyond the static report summary.

vi. Report History and Longitudinal Health Tracking

The platform maintains a complete history of all uploaded reports and their corresponding analyses for each patient. This report history module enables longitudinal tracking of biomarker values over time, allowing patients and doctors to observe health trends, monitor the effectiveness of lifestyle changes or treatments, and make more informed clinical decisions based on comparative data rather than a single snapshot.

vii. Notification System

DiagnoXpert includes a reliable email notification system implemented via Nodemailer. Patients and doctors receive timely alerts for key events such as report analysis completion, new consultation messages, and account activity ensuring that important health information is never missed.

viii. Security and Data Privacy

Patient data is protected through a layered security architecture. Passwords are hashed using bcryptjs, and session based authentication is managed through NextAuth.js. Role-based access control is enforced at both the frontend and API route levels, ensuring that patients and doctors' access only the data and features relevant to their roles. All uploaded files and media are stored

securely via Cloudinary with next cloudinary integration, and sensitive data is managed within a MongoDB database structured through Mongoose ODM.

ix. Deployment and CI/CD Pipeline

DiagnoXpert is deployed to Vercel, providing a globally distributed and scalable hosting environment optimized for Next.js applications. The CI/CD pipeline is managed through GitHub Actions, comprising two automated workflows: a continuous integration workflow that builds and verifies the application on Node.js 20.x and 22.x for every pull request, and a continuous deployment workflow that automatically ships the production build to Vercel upon every merge to the main branch.

x. Admin Panel and Doctor Verification

To maintain trust and quality across the platform, DiagnoXpert includes a dedicated Admin Panel that oversees doctor onboarding. When a doctor registers, their profile credentials, qualification, experience, and supporting documents enter a pending state and are not visible to patients until reviewed. The admin can inspect each submitted field individually, approving or rejecting specific details such as qualification or available days, and either verify the doctor's profile in full or reject it with the flagged issues sent back to the doctor for correction. The admin dashboard also provides a real time overview of platform activity, including total users, patients, doctors, and the current count of pending, verified, and rejected profiles. This verification layer ensures that only credentialed, approved doctors are able to participate in the Doctor Consultation Module.

2.3 Methodology and Software Lifecycle for This Project

DiagnoXpert was developed using a combined Agile Methodology and Iterative Software Process Model. This hybrid approach was selected to accommodate the inherent complexity and uncertainty of AI driven healthcare system development, where model performance evolves through testing and feedback, user requirements are refined through iterative usage, and clinical accuracy standards demand rigorous, incremental validation.

1.6.1 Agile Methodology

Agile development was adopted as the primary framework for managing the DiagnoXpert project. Under Agile, the overall system is decomposed into a series of sprints, each producing a tested and functional increment of the software. This approach offers several key advantages for an AI driven healthcare project.

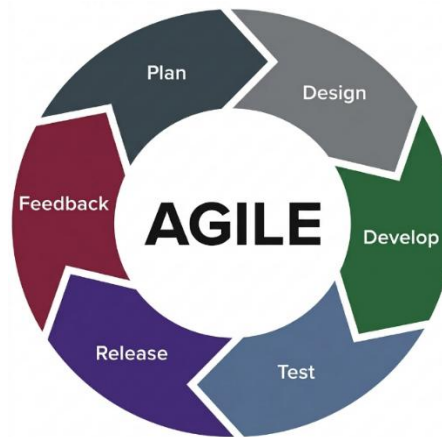


Figure 2.1 Agile Model Diagram

Medical data formats and laboratory report structures vary significantly across institutions and laboratories, requiring flexible and adaptive development practices. AI components including the OCR pipeline, biomarker parsers, and LLM analysis modules require frequent tuning and evaluation as new report samples become available. User requirements from patients, doctors, and administrative stakeholders may also evolve as the system matures and is exposed to real world usage scenarios.

The Agile model ensures that DiagnoXpert can evolve smoothly and incorporate improvements without disrupting the existing architecture. Sprint reviews and retrospectives provide structured opportunities to incorporate stakeholder feedback and adjust the development trajectory accordingly. Continuous integration via GitHub Actions and automated build verification ensure that new features do not introduce regressions in existing functionality.

1.6.2 Iterative Process Model

Alongside Agile, DiagnoXpert employs the Iterative Software Process Model, where the system is constructed through successive, increasingly capable versions. Each iteration delivers a fully tested and validated build with enhanced or newly introduced features. This is particularly critical in the medical domain, where accuracy and reliability are non-negotiable. The system was developed through the following iterative phases:

- **Iteration 1:** Core infrastructure user authentication via NextAuth.js, report upload handling via Multer, and basic OCR text extraction using Tesseract.js.
- **Iteration 2:** Biomarker parsing, abnormality detection, and health score computation.

- **Iteration 3:** LLM based analysis integration using Llama3-OpenBioLLM via the Hugging Face API for insight and recommendation generation.
- **Iteration 4:** Frontend dashboard development using Next.js 16, React 19, and Tailwind CSS including visualization modules and report history tracking.
- **Iteration 5:** Doctor Consultation Module and AI powered chatbot integration.
- **Iteration 6:** Testing, performance optimization, security hardening, and final deployment to Vercel via the CI/CD pipeline.

Rationale behind Selected Agile Model

The decision to adopt Agile was driven by the unique complexities, uncertainties, and evolving requirements inherent in building an AI powered healthcare platform that processes real world medical reports. In traditional waterfall approaches, all requirements must be defined and frozen at the outset. However, DiagnoXpert deals with highly variable medical data formats from different laboratories across Pakistan and South Asia some reports are clean digital PDFs while others are scanned documents with inconsistent layouts. AI components such as Tesseract.js OCR, biomarker extraction logic, and the Llama3-OpenBioLLM analysis pipeline require continuous tuning and improvement as new report samples and user feedback become available. Fixing all requirements upfront would have been impractical and introduced significant risk.

Key reasons for selecting Agile include:

- **High Adaptability and Flexibility:** Healthcare technology evolves rapidly. New medical guidelines, updated laboratory formats, improvements in OCR and LLM technologies, or shifts in user expectations can emerge at any time. Agile enables the team to respond to these changes within upcoming sprints rather than waiting for a complete system overhaul.
- **Frequent Delivery of Working Software:** Rather than delivering the entire system at the end of the project, Agile produces usable features after every sprint. This allowed early validation of core functions such as report upload, OCR extraction, and basic AI analysis, reducing the risk of building features that users ultimately find unnecessary or difficult to use.
- **Continuous Feedback and Improvement:** Sprint reviews and retrospectives provide structured opportunities to gather input from patients, doctors, and other stakeholders. This

feedback loop is especially valuable in a patient facing healthcare application where ease of use, clarity of explanations, and trust in AI generated insights are critical success factors.

- **Better Handling of AI-Specific Challenges:** AI models are probabilistic and improve with additional data and testing. Agile supports frequent evaluation and refinement of these components. Issues such as poor OCR accuracy on certain scanned PDFs or imprecise interpretations by the LLM can be detected and resolved in early sprints rather than becoming significant problems at a later stage.
- **Enhanced Cross Disciplinary Collaboration:** DiagnoXpert involves multiple disciplines AI integration, full-stack development with Next.js and TypeScript, UI/UX design with Tailwind CSS and shadcn/ui, and domain knowledge of medical reports. Agile promotes close collaboration through sprint planning and cross-functional team coordination, ensuring that technical decisions remain aligned with real medical needs and patient-friendly design principles.
- **Risk Reduction and Higher Quality:** In healthcare applications, accuracy, security, and reliability are non-negotiable. Agile's emphasis on continuous integration, automated testing through GitHub Actions, and incremental development helps identify bugs, security vulnerabilities, and performance issues early significantly lowering the risk of delivering an unreliable system that could mislead patients or compromise data privacy.
- **Faster Time to Value:** By delivering functional increments regularly, the project demonstrates visible and measurable progress. This enables partial deployment and pilot testing considerably earlier than a traditional development approach would allow.
- **Support for Future Compliance Needs:** Although DiagnoXpert is currently focused on patient education and health literacy rather than direct clinical diagnosis, Agile makes it straightforward to incorporate future compliance requirements such as healthcare data privacy standards or medical software guidelines incrementally without requiring a full restart of the project.

Overall, Agile was chosen because DiagnoXpert is not a simple static application. It is a complex, AI driven system operating in a dynamic healthcare environment with diverse users, variable data inputs, and high expectations for both accuracy and accessibility. The methodology ensures that

DiagnoXpert can evolve smoothly, remain aligned with real user needs, maintain high quality at every stage, and deliver meaningful value throughout the entire development lifecycle.

Chapter 3

Requirement Analysis

3 Requirement Analysis

The requirement analysis for DiagnoXpert focuses on identifying and defining the core features needed to build a simple, intelligent, and patient friendly healthcare platform. The system must support patient and doctor registration, secure role-based login, medical report upload in multiple formats, automated AI analysis using OCR and Large Language Models, generation of simple health summaries and recommendations, an interactive chatbot for health related questions, secure doctor-patient messaging, email notifications, and safe storage of all user data.

We used Use Case modeling as the main technique for analyzing these requirements. This approach clearly shows how the main users patients and doctors interact with the system to complete specific tasks. It helps visualize the full workflow, from registration and report upload, through AI analysis and chatbot queries, all the way to doctor consultation and secure data handling.

Use Case modeling helped us capture all the important features of the system, organize them in a clear and logical way, and build a strong foundation for the design and development phases. It also allowed us to prioritize the most important features and ensure that the platform remains easy to use for all types of users, including those with no technical or medical background.

The key system requirements identified through this analysis are summarized below.

- i. **User Registration and Profile Management:** Both patients and doctors can create accounts on the platform by providing basic details. Patients enter information such as name, age, gender, and medical history, while doctors provide their qualifications and specialization. The system automatically assigns the correct role to each user upon registration.
- ii. **Secure Role Based Login:** Users log in securely through NextAuth.js with passwords protected by bcryptjs hashing. The system enforces role based access control, ensuring that patients and doctors can only access the features and data that are relevant to their role.
- iii. **Medical Report Upload:** Patients can upload their diagnostic reports in PDF format. The system handles both digitally generated PDFs using pdf parse and pdfjs-dist, and scanned image based PDFs using Tesseract.js OCR with Sharp for image preprocessing.
- iv. **AI-Powered Report Analysis:** Once a report is uploaded and text is extracted, the system analyzes the biomarkers, detects abnormal values, and generates a clear, simple health

summary using the Llama3-OpenBioLLM model via the Hugging Face API. The analysis also includes a personalized health score and practical diet and lifestyle recommendations.

- v. **Interactive AI Chatbot:** Patients can ask health related questions through a built in chatbot and receive simple, context aware answers. This gives patients an easy way to understand their results and get guidance without waiting for a doctor.
- vi. **Doctor Consultation Module:** Patients can share their reports and AI generated analysis directly with a verified doctor on the platform. Doctors have a dedicated dashboard where they can review reports, add clinical notes, and communicate with patients through secure messaging.
- vii. **Notification System:** The platform sends email notifications to users for important events such as report analysis completion and new consultation messages. Notifications are handled through Nodemailer.
- viii. **Secure Data Storage:** All user data, reports, and analysis results are stored securely in a MongoDB database managed through Mongoose. Uploaded files are stored in the cloud via Cloudinary, and all access is protected through role based controls and encrypted authentication.

3. Use Case Diagram

In this Use case diagram, we show the behavior of our system. This enables you to visualize the different types of roles in a system and how those roles interact with the system. It outlines the various actions and functionalities that the system provides to its users.

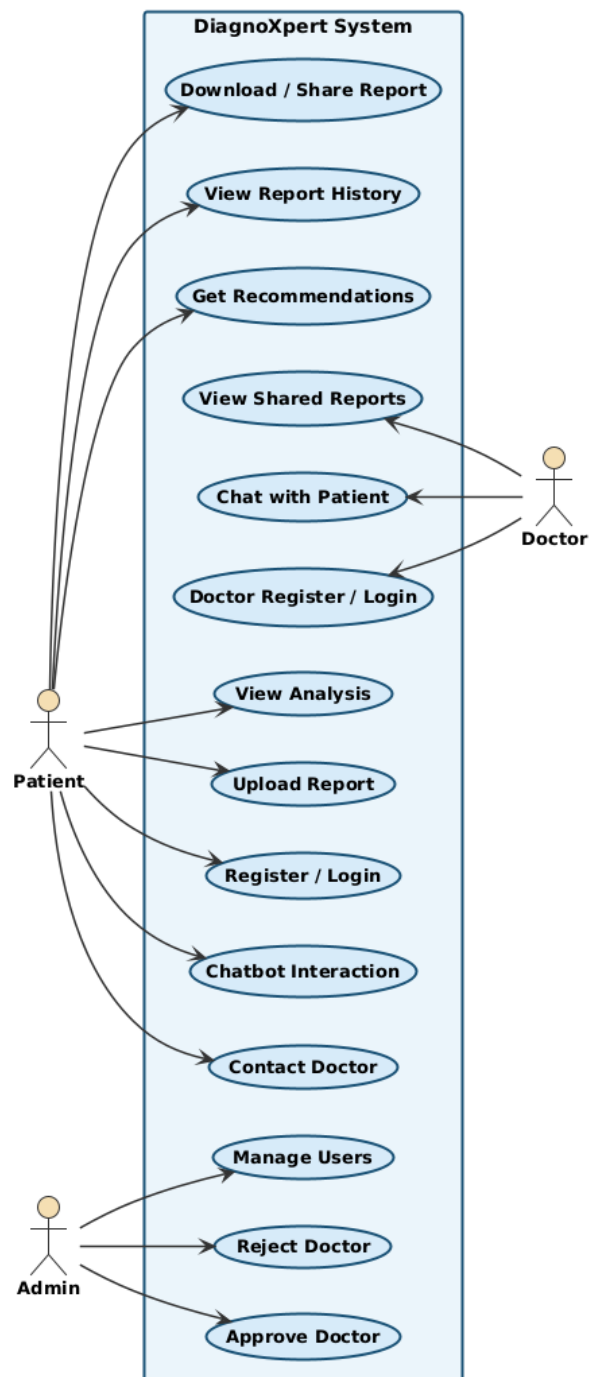


Figure 3.1 DiagnoXpert Use Case Diagram

3.1 Use Case Descriptions

The table below indicates comprehensive use case descriptions.

Table 3.1 Register/Login

Use Case ID	01
Use Case Name	Register/Login
Actors	Patient, Doctors
Description	The user (Patient or Doctor) creates a new account or logs into an existing account based on their role. The system uses NextAuth.js with role based access control to provide different features and permissions after successful authentication. Patients can upload medical reports, view OCR extracted data, analysis results, health insights, and report history. Doctors can view patient reports, add clinical notes, and provide professional recommendations.
Trigger	User selects the Register or Login option on the homepage.
Preconditions	User has a valid email address and internet access.
Post conditions	A new account is created with the correct role assigned, or an existing session is started with appropriate role based access rights.
Normal Flow	1. User opens the application and selects Register or Login. 2. User chooses their role: Patient or Doctor. 3. For registration, user enters required details name, email, phone, password, and role specific information. 4. For login, user enters registered email/phone and password. 5. System validates the information and sends OTP for verification. Upon successful validation, user is redirected to their role-specific dashboard.
Alternative Flow	If the users forget their password, the system provides a password reset mechanism via email using Nodemailer.

Table 3.2 Approve Doctor

Field	Detail
Use Case ID	02
Use Case Name	Approve Doctor
Actors	Admin
Description	The admin reviews a newly registered Doctor account and approves it after verifying credentials. Upon approval, the Doctor's role is activated in the system, granting them access to the Doctor dashboard, patient-shared reports, and the ability to chat with patients and provide clinical recommendations. Until approval, the Doctor account remains in a pending state and cannot access any patient data.
Trigger	A Doctor completes registration and their account enters the pending approval queue visible on the admin dashboard.
Preconditions	Admin is logged in with Admin level credentials. A Doctor has successfully submitted their registration with all required details (name, email, qualification, license number).
Post Conditions	The Doctor account status is updated to Approved. The Doctor receives an email notification confirming activation. The Doctor can now log in and access their role-specific dashboard.
Normal Flow	1. Admin logs in and navigates to the Doctor Management section. 2. Admin views the list of pending Doctor registration requests. 3. Admin selects a doctor entry and reviews submitted credentials and qualification details. 4. Admin clicks the Approve button. 5. System updates the Doctor's account status to Approved in the database. 6. System sends an automated email notification to the Doctor confirming account activation. 7. Doctor is now permitted to log in and access the Doctor dashboard.
Alternative Flow	If the Admin determines the Doctor's credentials are insufficient or suspicious, the admin rejects the application instead (see Reject Doctor use case). The Doctor is notified via email with the reason for rejection.

Table 3.3 Reject Doctor

Field	Detail
Use Case ID	03
Use Case Name	Reject Doctor
Actors	Admin
Description	The admin reviews a pending Doctor registration and rejects it if the submitted credentials do not meet the required standards, appear fraudulent, or are incomplete. The rejected account is flagged in the system and the Doctor is notified. The rejected Doctor cannot access the platform unless they re-register with corrected information.
Trigger	Admin reviews a pending Doctor account and determines it should not be approved.
Preconditions	Admin is logged in with Admin-level credentials. A Doctor registration is in the pending state awaiting Admin review.
Post Conditions	The Doctor account status is updated to Rejected. The Doctor receives an email notification informing them of the rejection. The account is locked and cannot access any Doctor-level features.
Normal Flow	1. Admin logs in and navigates to the Doctor Management section. 2. Admin views the list of pending Doctor registration requests. 3. Admin selects a doctor entry and reviews the submitted details. 4. Admin determines the credentials are invalid, incomplete, or unverifiable. 5. Admin clicks the Reject button and optionally enters a reason for rejection. 6. System updates the Doctor's account status to Rejected in the database. 7. System sends an automated email notification to the Doctor stating their application has been rejected along with the reason provided.
Alternative Flow	If the Admin accidentally clicks Reject, they may revert the decision by locating the rejected account and re-initiating an approval, provided the Doctor has not yet been notified.

Table 3.4 Upload Report

Use Case ID	04
Use Case Name	Upload Report
Actors	Patient
Description	The patient uploads their medical reports to the system so that the report can be processed through OCR, analyzed, and meaningful insights & recommendations can be generated.
Trigger	Patient selects Upload Report from the dashboard.
Preconditions	The Patient must be logged in with a valid account.
Post conditions	Report is successfully uploaded, stored securely, and forwarded for OCR extraction.
Normal Flow	1. Patient clicks on the Upload Report button. 2. System opens file explorer or drags and drop area. 3. Patient selects one or multiple report files from their device. 4. Patient optionally adds a title or description for the report. 5. System validates the file format and size. 6. File is uploaded to the server and stored securely. 7. System automatically triggers the OCR extraction process.
Alternative Flow	If unsupported format is selected then system prompts the patient to upload a valid file format.

Table 3.5 View Extracted Data

Use Case ID	05
Use Case Name	View Extracted Data
Actors	Patient
Description	The patient views the raw text and structured data extracted from the uploaded medical report through OCR and parsing. This allows the

	patient to verify the accuracy of the extracted information before analysis.
Trigger	A Patient selects View Extracted Data for a specific report.
Preconditions	Report must have been successfully uploaded and processed by the extraction pipeline.
Postconditions	Extracted text and structured biomarker data are displayed to the patient.
Normal Flow	1. Patient navigates to the specific report and clicks View Extracted Data. 2. System retrieves the OCR extracted text and parsed structured data. 3. Data is displayed in a clean tabular or key value format. 4. Patient can review the extracted values and highlight any incorrect entries. 5. Option to download the extracted data as JSON or PDF is available.
Alternative Flow	Patient may choose to download the extracted data instead of viewing it on screen.

Table 3.6 View Analysis

Use Case ID	06
Use Case Name	View Analysis
Actors	Patient
Description	The patient views a detailed, easy to understand analysis of their medical report. The analysis includes graphical representations, trend observations, and clear explanations of results in simple language.
Trigger	Patient selects View Analysis for a specific report.
Preconditions	AI analysis must have been successfully generated by the system.

Post conditions	Detailed analysis is displayed to the patient in both graphical and textual format.
Normal Flow	1. Patient clicks on View Analysis option for the desired report. 2. System retrieves the complete analysis data. 3. System displays results with color coded indicators. 4. Graphical charts are shown for better visualization. 5. Simple textual explanations and key insights are provided alongside graphs. 6. Doctor's notes are also displayed with the analysis.
Alternative Flow	Doctor may add professional notes that appear along with the system generated analysis.

Table 3.7 Doctor Consultation

Use Case ID	07
Use Case Name	Doctor Consultation
Actors	Patient, Doctor
Description	The patient shares their uploaded report and AI generated analysis with a verified doctor on the platform. The doctor reviews the report, adds clinical notes, and communicates with the patient through secure messaging.
Trigger	Patient selects Request Consultation from the report dashboard.
Preconditions	Patient must be logged in and have at least one analyzed report. Doctor must be registered and verified on the platform.
Post conditions	Consultation request is sent to the doctor. Doctor can review the report and respond with clinical notes or a message.
Normal Flow	1. Patient selects a report and clicks Request Consultation. 2. System displays a list of available verified doctors. 3. Patient selects a doctor and submits the consultation request. 4. Doctor receives a notification via email through Nodemailer.

	5. Doctor logs in, opens the consultation dashboard, and reviews the report and AI analysis. 6. Doctor adds clinical notes and sends a response message to the patient. 7. Patient receives a notification and views the doctor's response.
Alternative Flow	If no doctors are currently available, the system notifies the patient and allows them to send the request for later review.

Table 3.8 Chatbot Interaction

Use Case ID	08
Use Case Name	Chatbot Interaction
Actors	Patient
Description	The patient interacts with the AI powered chatbot to ask health related questions and receive context aware, plain language responses. The chatbot uses the Hugging Face API to generate relevant answers based on the patient's query and report context.
Trigger	Patient opens the chatbot from the dashboard or report view.
Preconditions	Patient must be logged in.
Post conditions	Patient receives a relevant and easy to understand response to their health query.
Normal Flow	1. Patient opens the chatbot interface. 2. Patient types or speaks a health related question. 3. System sends the query to the Hugging Face API. 4. AI generates a context aware, patient friendly response. 5. Response is displayed to the patient in the chat window.
Alternative Flow	If the API does not return a response within the expected time, the system displays a retry message.

Table 3.9 View Report History

Use Case ID	09
Use Case Name	View Report History
Actors	Patient, Doctor
Description	The patient or doctor views a complete history of all previously uploaded reports and their corresponding AI generated analyses. This enables longitudinal health trend tracking and comparison of biomarker values over time.
Trigger	Patient or Doctor selects Report History from the dashboard.
Preconditions	At least one report must have been previously uploaded and analyzed.
Post conditions	A chronological list of all past reports and analyses is displayed.
Normal Flow	1. User navigates to the Report History section. 2. System retrieves all past reports from MongoDB for the logged in user. 3. Reports are displayed in chronological order with key details. 4. User selects any report to view its full analysis. 5. Trend comparisons across multiple reports are displayed graphically.
Alternative Flow	If no previous reports exist, the system displays a prompt encouraging the patient to upload their first report.

Table 3.10 Manage Users

Field	Detail
Use Case ID	10
Use Case Name	Manage Users
Actors	Admin
Description	The admin has full oversight of all registered users on the DiagnoXpert platform, including Patients and Doctors. The admin can view user profiles, monitor account activity, suspend or deactivate accounts that

	violate platform policies, and permanently delete accounts when required. This use case ensures platform integrity, data governance, and compliance with privacy policies.
Trigger	Admin navigates to the User Management section of the Admin dashboard.
Preconditions	Admin is authenticated and has Admin-level access rights. At least one registered user account exists in the system.
Post Conditions	The selected user account is updated, suspended, or deleted as per Admin action. Affected users receive appropriate email notifications regarding changes to their account status.
Normal Flow	1. Admin logs in and opens the User Management panel. 2. System displays a list of all registered users (Patients and Doctors) with their status, role, and registration date. 3. Admin searches or filters users by name, email, role, or status. 4. Admin selects a specific user to view their full profile and activity. 5. Admin performs one of the following actions: a. Suspend Account temporarily disables the user's access or Reactivate Account restores access to a previously suspended account; or Delete Account permanently removes the user and their associated data from the system. 6. System updates the database to reflect the admin's action. 7. System sends an automated notification to the affected user.
Alternative Flow	If the Admin attempts to delete an account that has active shared reports or ongoing doctor-patient consultations, the system displays a warning prompt asking the Admin to confirm the deletion or resolve pending activity first before proceeding.

3.2 Functional Requirements

Functional requirements describe what the software must do and how it should behave in various situations. These requirements were documented during the early stages of the development lifecycle and serve as the foundation for system design and implementation.

4.2.1 User Authentication and Authorization The system shall allow patients and doctors to register and log in securely, while admin accounts are provisioned separately for platform oversight. Role based access control ensures that patients, doctors, and admins can only access their respective dashboards and data.

4.2.2 Report Upload System The system shall allow patients to upload medical reports in PDF format. Users can select and upload files through a user friendly drag and drop interface with progress indicators and file format and size validation handled by Multer. Uploaded files are stored securely in Cloudinary via next cloudinary integration. Patients can also share specific reports with chosen doctors, with the ability to grant or revoke access at any time.

4.2.3 OCR and Document Extraction System The system shall extract medical text data from uploaded reports using Optical Character Recognition and PDF parsing. For digitally generated, text based PDFs, text extraction is performed using pdf parse and pdfjs-dist. For scanned or image based PDFs, Tesseract.js OCR is used in combination with Sharp for image preprocessing. The extraction pipeline accurately identifies and retrieves structured medical information such as test names, values, reference ranges, and dates from a wide variety of report formats.

4.2.4 Abnormality Detection and Highlighting The system shall automatically detect and highlight abnormal results with severity classification. After processing a report, the system compares extracted biomarker values against standard clinical reference ranges and classifies each result as normal, borderline, or abnormal. Results are displayed with color coded indicators and severity labels to help patients quickly identify areas of concern.

4.2.5 Health Score System The system shall compute and display a personalized health score based on report data. A composite health score is calculated using multiple biomarker parameters from the uploaded report, weighted medical indicators, and where available, trends observed across multiple reports over time. This gives patients a simple and meaningful overview of their overall health status.

4.2.6 Disease Prediction and Risk Analysis The system shall identify possible health risks and associated contributing factors based on the extracted report data. Using the Llama3-OpenBioLLM model via the Hugging Face API, the system suggests potential conditions or risk areas along with contextual explanations. All predictions are accompanied by clear disclaimers stating that the output is for informational purposes only and does not substitute professional medical diagnosis.

4.2.7 AI Generated Health Insights and Recommendations The system shall generate plain language explanations of report findings along with personalized diet and lifestyle recommendations. The Llama3-OpenBioLLM model transforms complex clinical data into simple, patient friendly summaries that help users understand their results and take practical steps to improve their health.

4.2.8 Report History and Longitudinal Tracking The system shall maintain a complete history of all uploaded reports and their analyses for each user. Patients and doctors can access a chronological, filterable, and searchable view of all past reports, with options to sort by date or test type. Trend comparisons across multiple reports are displayed graphically to support longitudinal health monitoring.

4.2.9 Doctor Consultation Module The system shall allow patients to request consultations with verified doctors registered on the platform. Patients can browse available doctors, view their profiles, and send consultation requests along with their uploaded reports and AI-generated analysis. Doctors have a dedicated dashboard to review reports, add clinical notes, and communicate with patients through secure messaging. Patients can browse available doctors limited to those verified by the admin view their profiles, and send consultation requests

4.2.10 AI Powered Chatbot The system shall provide an AI-powered chatbot for health related queries and report guidance. The chatbot allows patients to ask questions in natural language typed or spoken and receive context aware, patient friendly responses generated through the Hugging Face API. It assists users with understanding report findings, clarifying medical terms, and navigating the platform.

4.2.11 Notification System The system shall send timely notifications to users for key events including report analysis completion, new consultation messages, and account activity. Notifications are delivered via email using Nodemailer, ensuring that patients and doctors stay informed without needing to remain active on the platform.

4.2.12 Report Generation and Download The system shall allow users to generate and download professional summaries of their report analysis. Downloadable outputs include analysis summaries, health scores, highlighted abnormalities, risk predictions, and recommendations formatted for personal records or use during clinical consultations.

4.2.13 Admin Panel and Doctor Verification The system shall provide an admin panel for reviewing and managing doctor registrations. When a doctor registers, their profile, credentials, and supporting documents (medical certificate, ID card) enter a pending state until reviewed. The admin can inspect each submitted field individually, approve or reject specific details, and either verify the doctor's profile in full or reject it with the flagged issues returned to the doctor for correction. The admin dashboard also displays real-time platform statistics, including total users, patients, doctors, and the current counts of pending, verified, and rejected profiles.

3.3 Non-functional Requirements

Non-functional requirements define the quality attributes and operational constraints of the system. The following non-functional requirements have been identified for DiagnoXpert.

4.3.1 Performance The system shall respond to user requests within 2 seconds for standard operations such as login, dashboard loading, and report browsing. OCR processing using Tesseract.js and AI based analysis via the Hugging Face API shall complete within 10 to 30 seconds depending on the complexity and size of the uploaded report. Next.js 16 with Turbopack is used to minimize build times and optimize server side rendering performance.

4.3.2 Scalability The system shall be designed to support up to 10,000 active users and handle at least 1,000 report uploads and analyses per day. The deployment on Vercel provides automatic horizontal scaling to accommodate future growth in users and data volume without requiring manual infrastructure changes. Cloudinary handles media storage scaling independently of the application server.

4.3.3 Security The system shall ensure data confidentiality, integrity, and availability at all times. All sensitive medical data shall be encrypted in transit via HTTPS. Passwords are hashed using bcryptjs and user sessions are managed securely through NextAuth.js. Role based access control is enforced at both the frontend and API route levels across all three roles patient, doctor, and admin ensuring each user can only access functionality and data relevant to their role

4.3.4 Reliability and Availability The system shall achieve at least 99.5% uptime, supported by Vercel's globally distributed deployment infrastructure. Automatic backups of the MongoDB database shall be performed regularly, with a recovery point objective of less than 24 hours and a recovery time objective of less than 4 hours in the event of data loss or system failure.

4.3.5 Usability The user interface shall be intuitive, responsive, and accessible across desktops, tablets, and mobile browsers. Built with Tailwind CSS v4 and shadcn/ui, the interface follows clean design principles that ensure ease of navigation for users of all technical backgrounds. The system shall support both English and Urdu languages, and all report explanations shall be presented in plain, non-technical language to serve users with basic digital literacy.

4.3.6 Maintainability The system architecture shall be modular and well documented to allow straightforward maintenance, updates, and addition of new features. The Next.js full-stack codebase is written in TypeScript, ensuring type safety and reducing runtime errors. Code follows standard practices with proper logging, error handling, and component-based structure. The CI/CD pipeline via GitHub Actions ensures that all changes are automatically tested and deployed without manual intervention.

4.3.7 Data Accuracy The OCR extraction pipeline using Tesseract.js shall achieve at least 90% accuracy on standard printed medical reports. The AI analysis generated by Llama3 OpenBioLLM shall provide reliable and contextually appropriate outputs. All system generated insights shall be accompanied by clear disclaimers stating that results are for informational purposes only and do not substitute professional medical advice.

4.3.8 Privacy Patients retain full control over report sharing and may revoke access at any time. The system shall not use, share, or process personal health information for advertising or any unauthorized purpose. All data handling practices are designed in accordance with healthcare data privacy principles. Admin access is limited strictly to doctor verification data credentials, qualifications, and submitted documents and does not extend to patient medical reports, analysis results, or consultation content.

4.3.9 Backup and Recovery All user data, uploaded reports, and system configurations shall be automatically backed up on a regular schedule. MongoDB Atlas compatible backup strategies shall support point in time recovery in the event of data loss or corruption. Cloudinary provides independent redundant storage for all uploaded media files.

4.3.10 Compliance The system shall comply with all applicable laws and regulations related to electronic health records, data protection, and medical software in Pakistan. The platform is built with privacy-first design principles and is structured to accommodate future compliance requirements including potential alignment with international healthcare data standards through the incremental nature of its Agile development process.

Chapter 4

Design and Architecture

4 Design and Architecture

4.1 System Architecture

DiagnoXpert employs a layered, modular architecture structured into six principal layers: the Client Layer, Frontend Layer, Backend Layer, Processing Layer, Storage Layer, and CI/CD Layer. This separation of concerns ensures that each layer can be developed, tested, and scaled independently, while Next.js API Routes serve as the central communication hub coordinating all inter-layer interactions.

The **Client Layer** consists of a standard web browser through which all users' patients, doctors, and the admin access the platform with no installation required.

The **Frontend Layer** is built with Next.js, React, TypeScript, Tailwind CSS v4, and shaden/ui. Redux Toolkit manages global application state, ensuring consistent UI behaviour across the platform.

The **Backend Layer** is implemented as Next.js API Routes and acts as the system's central orchestrator. It exposes four dedicated services. The Authentication service handles user identity verification and session management using NextAuth.js with role-based access control enforced at the API level. The Admin Service manages doctor approval and rejection workflows along with overall user account management. The Notification Service dispatches automated email alerts to users such as registration confirmations and doctor status updates via Nodemailer. The File Handler manages upload validation and routes report files to both the Storage Layer and the Processing Layer accordingly.

The **Processing Layer** operates as a three-stage pipeline triggered on every report upload. The OCR Engine extracts text from PDFs using pdf-parse or from scanned images using Sharp and Tesseract.js. The Text Parser then identifies biomarker parameters, values, and reference ranges. Finally, the AI Model Llama3-OpenBioLLM via the Hugging Face Inference SDK generates plain-language explanations, detects abnormalities, computes a health score, and produces lifestyle recommendations.

The **Storage Layer** uses MongoDB with Mongoose ODM for all structured data and Cloudinary for secure cloud storage of uploaded report files.

The **CI/CD Layer** uses GitHub Actions for continuous integration on pull requests and continuous deployment to Vercel on every merge to the main branch, ensuring the live platform always reflects the latest verified codebase.

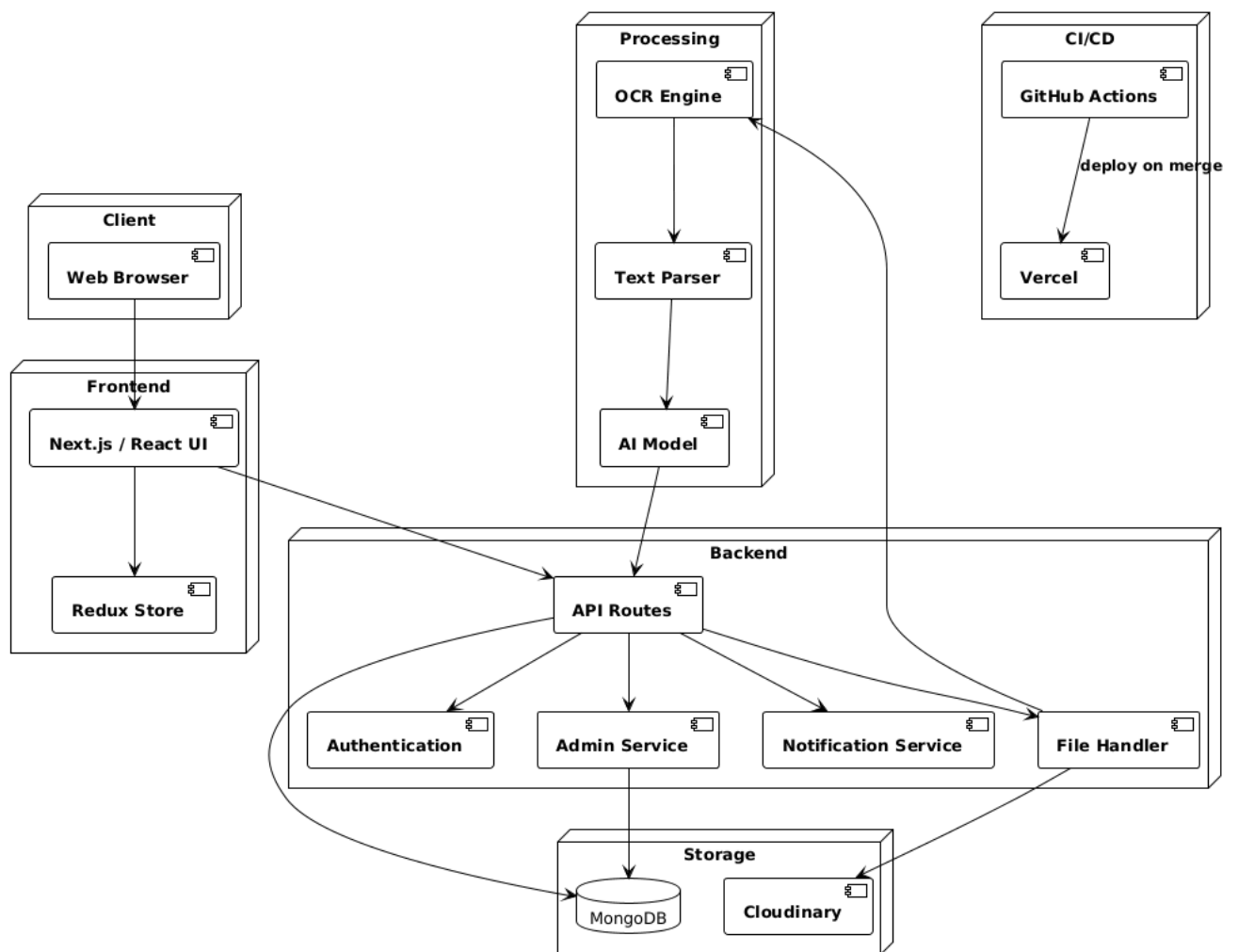


Figure 4.1 DiagnoXpert System Architecture

System Architecture Explanation

The architecture diagram above illustrates the overall workflow and component interactions within the DiagnoXpert web application. The system is designed to provide AI powered medical report analysis, secure doctor-patient consultation, longitudinal health tracking, and reliable data management through a clean, modular, and scalable structure.

At the core of the system is the **Client Layer**, which represents the web browser used by all platform users. Patients and doctors' access DiagnoXpert entirely through a browser based

interface with no installation required. The browser communicates with the Frontend Layer to render the user interface and dispatches requests to the backend through Next.js API Routes.

The **Frontend Layer** is the user-facing component of the application, built with Next.js and React. It renders all pages and UI components, manages routing, and handles user interactions. The Redux Store maintains global application state such as the currently logged-in user, uploaded report status, and consultation data ensuring a consistent experience as users navigate between sections of the platform. Tailwind CSS v4 and shadcn/ui provide the visual styling and component structure. The frontend communicates bidirectionally with the Backend API Routes, sending user actions and receiving processed responses.

The **Backend Layer** is implemented entirely through Next.js API Routes and acts as the central orchestrator of the entire system. When a user performs an action such as uploading a report, requesting an analysis, or sending a consultation message the frontend dispatches an HTTP request to the appropriate API route. The backend validates the request, enforces role-based access control, and coordinates with all other layers to return a structured response. Four dedicated services branch from the API Routes: the Authentication service handles user identity verification and session management via NextAuth.js; the Admin Service manages doctor approval, user oversight, and platform governance; the Notification Service dispatches email alerts and system notifications to users via Nodemailer; and the File Handler manages the receipt, validation, and routing of uploaded report files before forwarding them to Cloudinary for storage and to the Processing Layer for analysis.

The **Processing Layer** is triggered by the backend whenever a report is uploaded and consists of three sequential components. First, the OCR Engine handles text extraction from uploaded documents. For digitally generated PDFs, pdf-parse and pdfjs-dist extract raw text content directly. For scanned or image-based reports, Sharp first preprocesses the image to improve quality through deskewing, noise reduction, and contrast enhancement, after which Tesseract.js performs optical character recognition to extract the text. The extracted raw text is then forwarded to the Text Parser, which identifies individual biomarker parameters, their measured values, units, and reference ranges from the unstructured content. The parsed and structured data is finally passed to the AI Model Llama3-OpenBioLLM accessed via the Hugging Face Inference SDK which generates plain-language explanations, detects abnormalities, computes a personalized health score, and produces diet and lifestyle recommendations. All results are returned to the Backend API Routes for storage and delivery to the user.

The **Storage Layer** consists of two components working in parallel, both connected to the Backend API Routes. MongoDB, managed through Mongoose ODM, stores all structured application data including user profiles, report metadata, AI generated analysis results, consultation records, and chat history. Cloudinary, integrated via next-cloudinary, handles the storage of all uploaded report files and associated media, providing secure, scalable, and redundant cloud file management independently of the application server. The File Handler routes uploaded files directly to Cloudinary, while analysis results and metadata flow from the API Routes into MongoDB.

The **CI/CD Layer** automates the build, verification, and deployment process independently of the application runtime. Every pull request triggers the continuous integration workflow on GitHub Actions, which builds the Next.js application to verify code quality and catch errors before merging. Upon every successful merge to the main branch, GitHub Actions automatically triggers the continuous deployment workflow, which builds and ships the production application to Vercel as indicated by the "deploy on merge" arrow in the diagram. This eliminates manual deployment steps and ensures the live platform always reflects the latest verified codebase.

Together, these layers form a cohesive, scalable, and secure system that takes a patient's raw medical report as input and delivers clear, actionable health insights while maintaining the privacy, reliability, and accessibility standards required of a patient facing healthcare platform.

4.2 Process Flow/Representation

The flowchart below illustrates the complete logical flow of the DiagnoXpert web application, showing how all three user types Patients, Doctors, and Admin interact with the system from initial registration through to report analysis, consultation, and health tracking.

i. Registration and Login

The process begins when a user opens DiagnoXpert in their browser and is prompted to either register or log in. The system validates the submitted credentials. If the credentials are invalid, the user is redirected back to the login screen. If valid, the system checks the user's assigned role and branches into one of three flows accordingly.

ii. Admin Flow

If the authenticated user's role is Admin, they are directed to the Admin Panel. The admin's primary responsibility is managing doctor registrations. For each pending doctor account, the admin makes a decision to either approve or reject. If approved, the system activates the doctor account. If rejected, the doctor account is rejected and marked accordingly. In both cases, the system automatically notifies the doctor via email about the outcome.

iii. Doctor Flow

If the authenticated user's role is Doctor, the system first checks whether the Admin has approved their account. If not yet approved, access is denied and the doctor cannot proceed. If approved, the doctor is granted access to view shared patient reports. From there, the doctor can review the AI generated analysis and send a consultation reply to the patient.

iv. Patient Flow

If the authenticated user's role is Patient, they are directed to upload a medical report. The system first checks whether the uploaded file is in a valid format. If the format is invalid, a format error is shown and the process ends. If the format is valid, the system determines the file type. For PDF files, text is extracted directly from the document. For image based files, the system runs OCR on the image to extract the text.

After extraction, the system checks whether the extraction was successful. If extraction fails, the patient is prompted to re-upload the report. If extraction succeeds, the extracted content is passed to the AI model, which analyzes the biomarker data, computes a personalized health score, and generates a summary along with diet and lifestyle recommendations. All results are saved to the database and displayed to the patient on their Analysis Dashboard.

From the Analysis Dashboard, the patient has three options available in parallel. They can view their AI generated recommendations, download their report, or contact a doctor. If the patient contacts a doctor, the doctor reviews the report and sends a reply, after which the patient is notified. All three paths converge at the end of the flow, concluding the patient's session.

Common System Functions Across All Roles

Throughout all interactions, Next.js API Routes handle all backend communication. MongoDB with Mongoose ODM persists all structured data including user profiles, report records, analysis results, and consultation history. Cloudinary manages secure cloud storage for all uploaded report files. Nodemailer delivers email notifications for key events such as doctor approval, rejection, and consultation replies. The Hugging Face Inference SDK provides LLM inference for report analysis and the AI chatbot. GitHub Actions automates continuous integration and deployment, ensuring the live platform on Vercel always reflects the latest verified build.

Together, these flows form a coherent end-to-end system that takes a patient's raw medical report as input and delivers clear health insights, professional consultation, and reliable data management all within a single secure and accessible web platform.

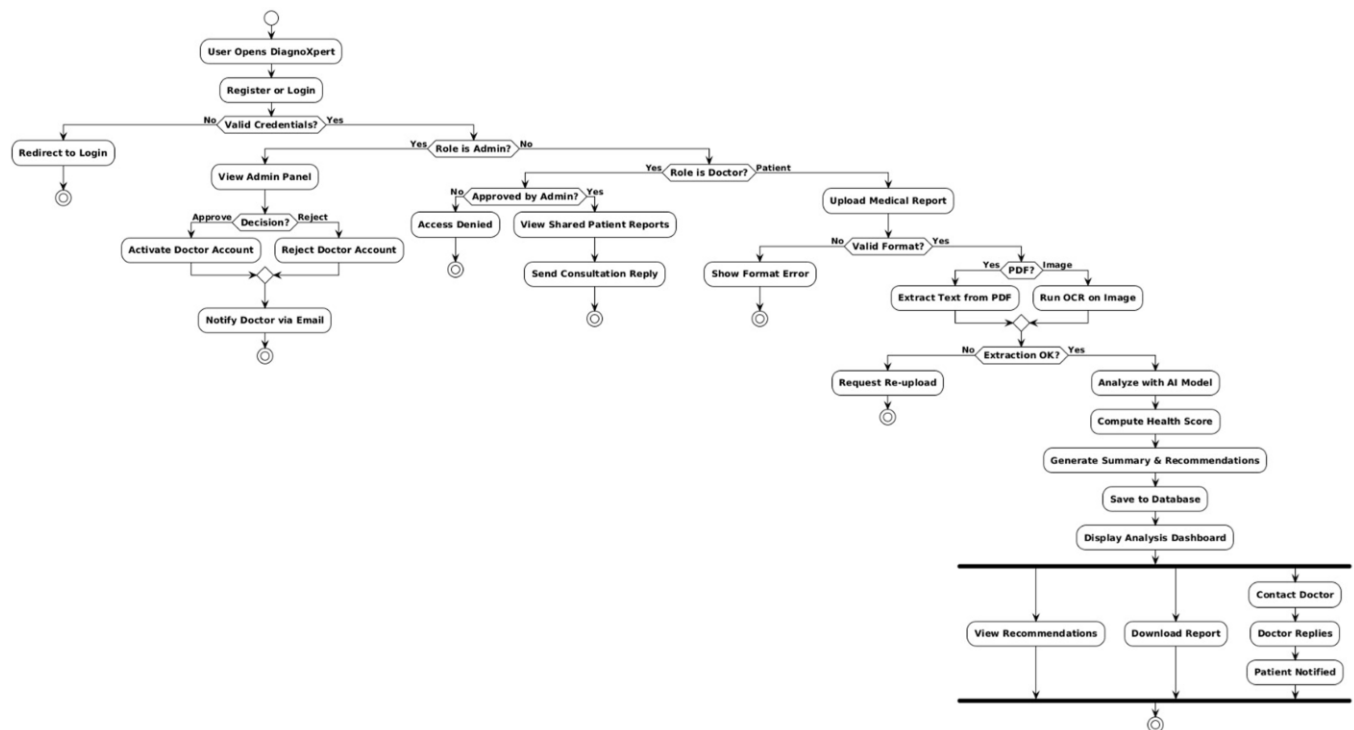


Figure 4.2 DiagnoXpert Process Flow

4.3 Sequence Diagram

The sequence diagram below illustrates the chronological flow of interactions between the three primary actors Patient, Doctor, and Admin and the core system components: Frontend, API, Auth, File Handler, OCR Engine, AI Model, and Database. The diagram is divided into three sections: Doctor Registration and Approval, Patient Login and Report Analysis, and Consultation.

i. Doctor Registration and Approval

The sequence begins with the Doctor registering on the platform by submitting their credentials through the Frontend. The Frontend sends the registration request to the API, which saves the doctor record to the Database with a status of pending. The Database confirms the save, and the Doctor is notified that their account is awaiting Admin approval.

The admin then logs in to the Admin Panel. The Frontend fetches the list of pending doctors from the API, which queries the Database for all pending registrations. The Database returns the pending list, and the Frontend displays it to the Admin. The admin reviews the entries and submits an approve or reject decision. The API receives the decision and updates the doctor's status in the Database accordingly. The Doctor then receives an email notification informing them whether their account has been approved or rejected.

ii. Patient Login and Report Analysis

The patient enters their credentials on the login screen. The Frontend sends a login request to the API, which forwards the credentials to the Auth module for validation. Auth validates the credentials and issues a session token back to the API. The API returns a login success response to the Frontend, and the patient's dashboard is displayed.

The patient then uploads a medical report. The Frontend sends the file to the API, which forwards it to the File Handler for processing. The File Handler passes the file to the OCR Engine, which extracts the raw text from the document and returns it to the File Handler. The extracted text is sent back to the API, which then forwards the structured data to the AI Model. The AI Model analyzes the biomarkers, computes the health score, and generates a summary with recommendations, returning the analysis result to the API. The API saves both the report and the complete analysis to the Database. Once saved, the API notifies the Frontend that the analysis is ready, and the results are displayed to the patient.

iii. Consultation

After reviewing their results, the patient initiates a consultation by selecting Contact Doctor. The Frontend sends a consultation request to the API, which stores the request in the Database and simultaneously dispatches an email notification to the Doctor. The Doctor logs in and views the shared patient reports through the Frontend. The Doctor reviews the AI generated analysis and submits a reply. The Frontend sends the reply to the API, which saves it to the Database. The patient is then notified of the doctor's response and can view it directly on their dashboard.

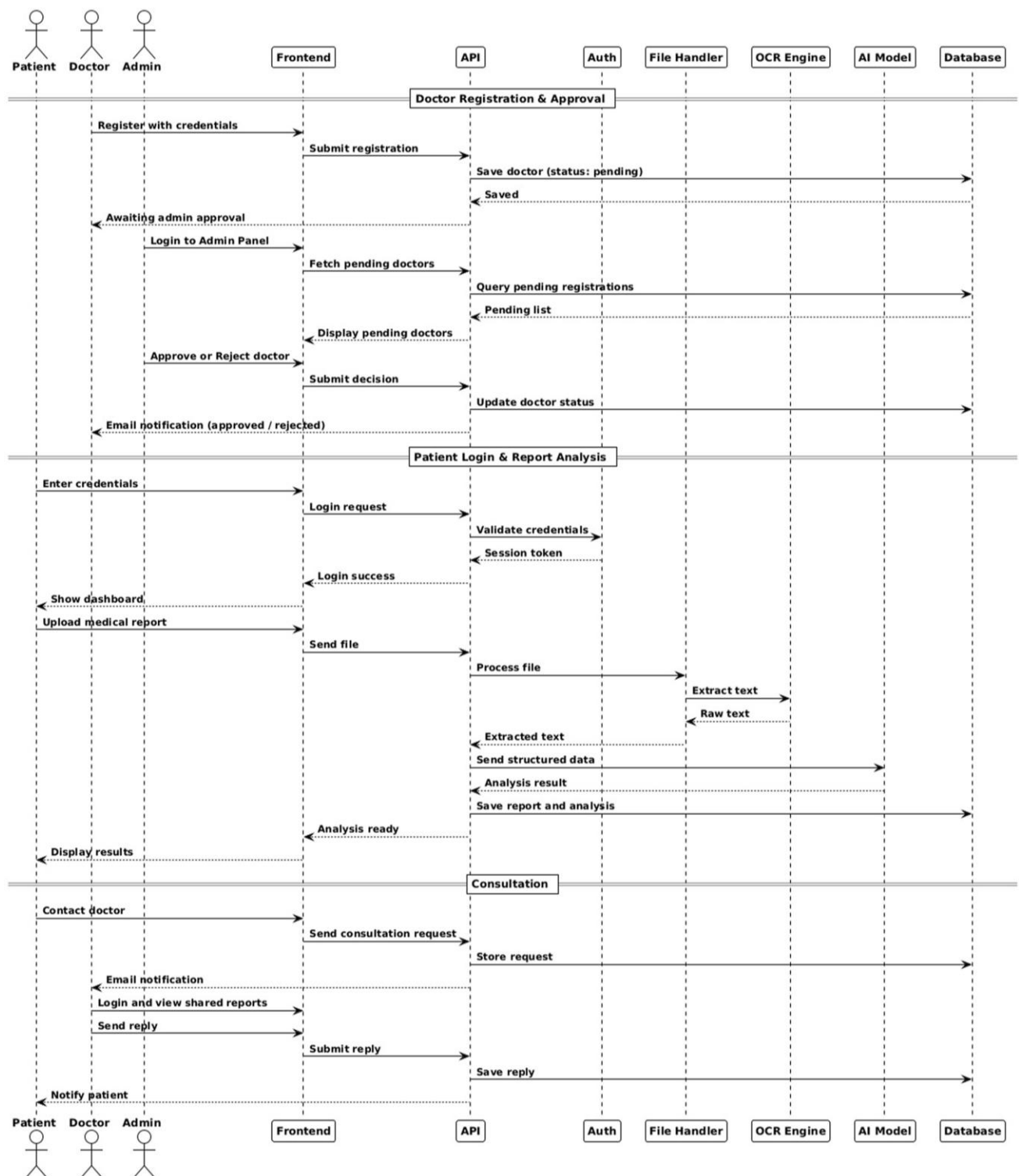


Figure 4.3 DiagnoXpert Sequence Diagram

4.4 Class Diagram

The class diagram below defines the core data structures, attributes, methods, and relationships that form the object-oriented foundation of the DiagnoXpert system. Each class maps directly

to a MongoDB collection managed through Mongoose ODM, and the relationships between classes reflect the actual data associations enforced at the application level.

i. User Class

The User class sits at the top of the hierarchy and serves as the base for all platform participants. It stores the common attributes shared by every account: id, name, email, password, and role. The password field stores the bcryptjs-hashed value rather than plain text. The role attribute determines which dashboard and features the user can access after authentication via NextAuth.js. The class exposes three core methods: register(), login(), and logout().

ii. Patient, Doctor, and Admin Classes

Patient, Doctor, and Admin all inherit from the User class through an inheritance relationship, each extending it with role-specific attributes and methods.

The **Patient class** adds dateOfBirth and gender, and exposes four methods: uploadReport(), viewAnalysis(), viewHistory(), and contactDoctor(), reflecting the complete set of actions available to a patient on the platform.

The **Doctor class** adds license, specialization, and approvalStatus attributes. The approvalStatus field reflects whether the doctor's account is pending, approved, or rejected. It exposes two methods: viewSharedReports() and sendReply(), representing the doctor's ability to review patient reports and respond with clinical feedback.

The **admin class** exposes four methods: viewPendingDoctors(), approveDoctor(), rejectDoctor(), and manageUsers(). The admin holds a direct approves/rejects relationship with the Doctor class, meaning the admin is responsible for updating the doctor's approvalStatus. This gatekeeping ensures that only verified doctors can access patient data on the platform.

iii. Report Class

A Patient uploads zero or more Reports, represented by a one-to-many relationship. The Report class stores the file's id, fileUrl pointing to its Cloudinary location, fileType indicating whether it is a digital or scanned PDF, status reflecting the current processing state, and uploadedAt timestamp. It exposes upload() and getStatus() methods, triggered when a patient submits a file through the upload pipeline.

iv. Analysis Class

Each Report generates exactly one Analysis, represented by a one-to-one relationship. The Analysis class stores the healthScore computed by the scoring engine, a list of abnormalities detected through biomarker comparison, a plain-language summary generated by the Llama3-

OpenBioLLM model, and a createdAt timestamp. It exposes compute() and getSummary() methods. This class represents the core output of the AI/ML processing pipeline.

v. Recommendation Class

Each Analysis produces exactly one Recommendation through a one-to-one relationship. The Recommendation class stores a list of suggestions covering diet, lifestyle, and follow-up actions generated by the LLM. It exposes a single generate() method, called as the final step of the analysis pipeline before results are displayed to the patient.

vi. Consultation Class

A Patient initiates zero or more Consultations, and a Doctor responds to zero or more Consultations, both represented by one-to-many relationships. The Consultation class stores the message sent by the patient, the reply provided by the doctor, the current status of the consultation, and a createdAt timestamp. It exposes send() and reply() methods, supporting the full doctor-patient communication workflow within the platform.

vii. Notification Class

Each Consultation triggers a Notification. The Notification class stores the message content and type, and exposes a send() method implemented through Nodemailer. This class ensures that both patients and doctors are informed of key events via email without needing to remain active on the platform.

viii. Summary of Relationships

The class structure reflects a clear separation of concerns. User is the common base extended by Patient, Doctor, and Admin. The admin governs platform integrity by approving or rejecting Doctor accounts before they can access any patient data. Patient drives the core workflow by uploading Reports, which generate Analysis objects, which in turn produce Recommendations. Separately, Patient initiates Consultations with Doctors, and each Consultation triggers a Notification. This modular design ensures that changes to one component do not disrupt the rest of the system, and provides a solid foundation for future expansion including additional report types, extended doctor profiles, or integration with hospital EMR systems.

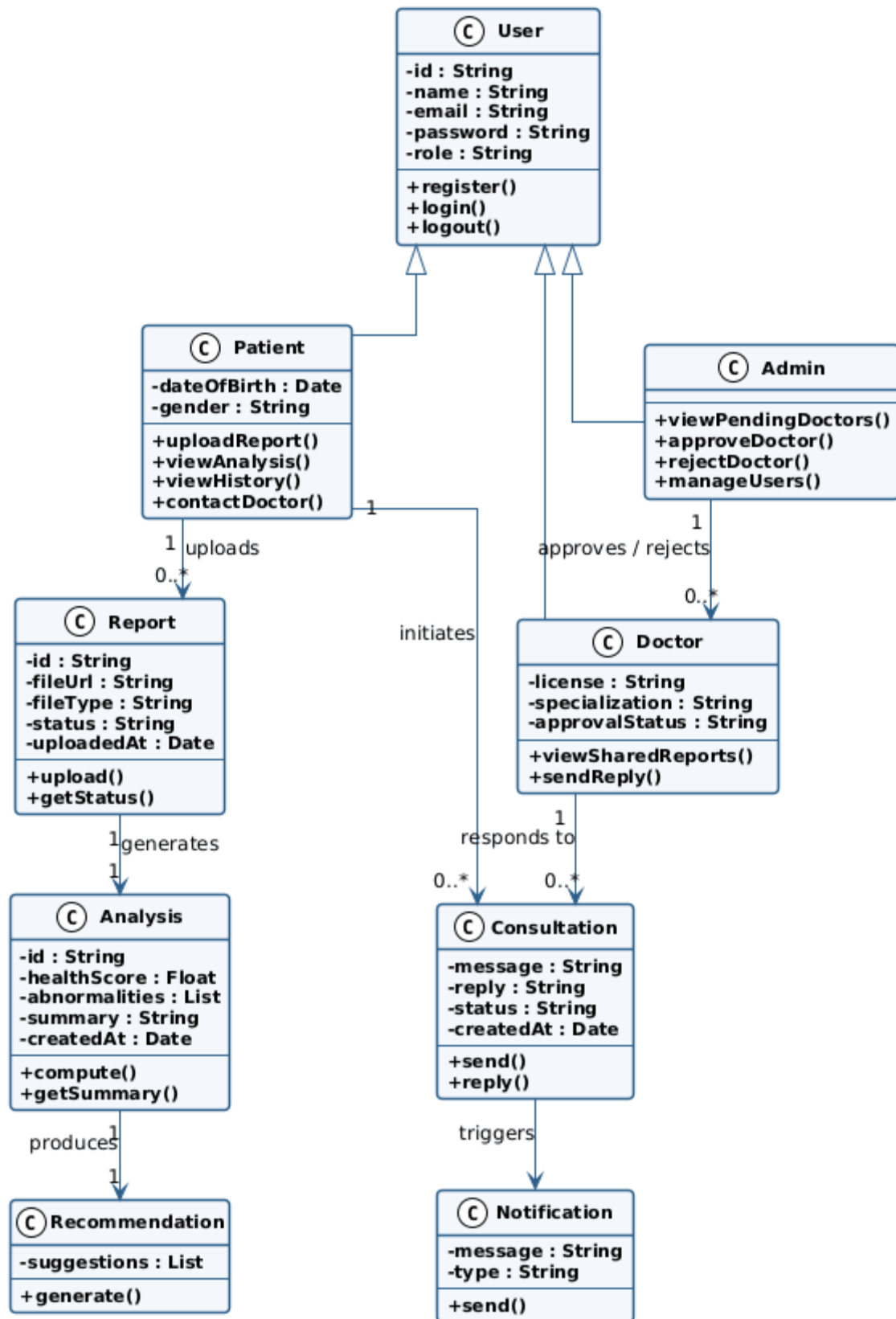


Figure 4.4 DiagnoXpert Class Diagram

4.5 Activity Diagram

The activity diagram below illustrates the complete end-to-end workflow of DiagnoXpert across four swim lanes Patient, System, Doctor, and Admin showing how actions and decisions flow between users and the platform from login through to report history, doctor consultation, and admin governance

i. Authentication

The process begins when the Patient opens DiagnoXpert and enters their login credentials. The System validates the credentials through NextAuth.js. If the credentials are invalid, the System displays an error message and the Patient is prompted to re-enter their login credentials. The System validates again, and upon success starts a session, allowing the Patient to proceed to their dashboard.

ii. Report Upload and Validation

Once on the dashboard, the Patient uploads a medical report. The System checks whether the file format is valid. If the format is not supported, the file is rejected with a format error and the process ends. If valid, the file is stored securely and the System determines the file type. If it is a digital PDF, text is extracted directly using pdf-parse and pdfjs-dist. If it is a scanned image-based file, Sharp preprocesses the image and Tesseract.js runs OCR to extract the text. Both paths merge at an extraction check. If extraction fails, the System notifies the patient that extraction failed and the process ends. If extraction is successful, the flow continues to the processing pipeline.

iii. AI Processing Pipeline

The System parses the extracted raw text into structured biomarker data. The AI analysis is then run using the Llama3-OpenBioLLM model via the Hugging Face API. The System detects abnormalities and classifies their severity, computes a personalized health score, and generates diet and lifestyle recommendations. All results are saved to the Database and the Patient is directed to their Analysis Dashboard.

iv. Patient Actions

From the Analysis Dashboard, three parallel actions become available. The Patient can view their personalized recommendations, download the report, or send a message to a doctor to request a consultation. All three paths merge after completion and the Patient can then view their full Report History, concluding the patient flow.

v. Doctor Consultation

When the Patient sends a consultation message to a doctor, the Doctor receives a notification. The Doctor reviews the shared report and AI-generated analysis and sends a consultation reply. The System delivers the reply back to the Patient, who receives the doctor's response on the platform.

vi. Admin Flow

In parallel with the patient and doctor flows, the Doctor registers on the platform. The admin views the list of pending doctor registrations through the Admin Panel. The admin then makes a decision for each pending entry. If approved, the doctor account is activated. If rejected, the doctor account is rejected. Both outcomes merge and the System sends an email notification to the doctor informing them of the decision. The admin then manages users and monitors the overall system, concluding the admin activity.

vii. Report History

After all actions are completed whether the patient simply viewed results, downloaded their report, or completed a doctor consultation the System updates the patient's report history in MongoDB. The patient can then view their full longitudinal report history, and the activity concludes.

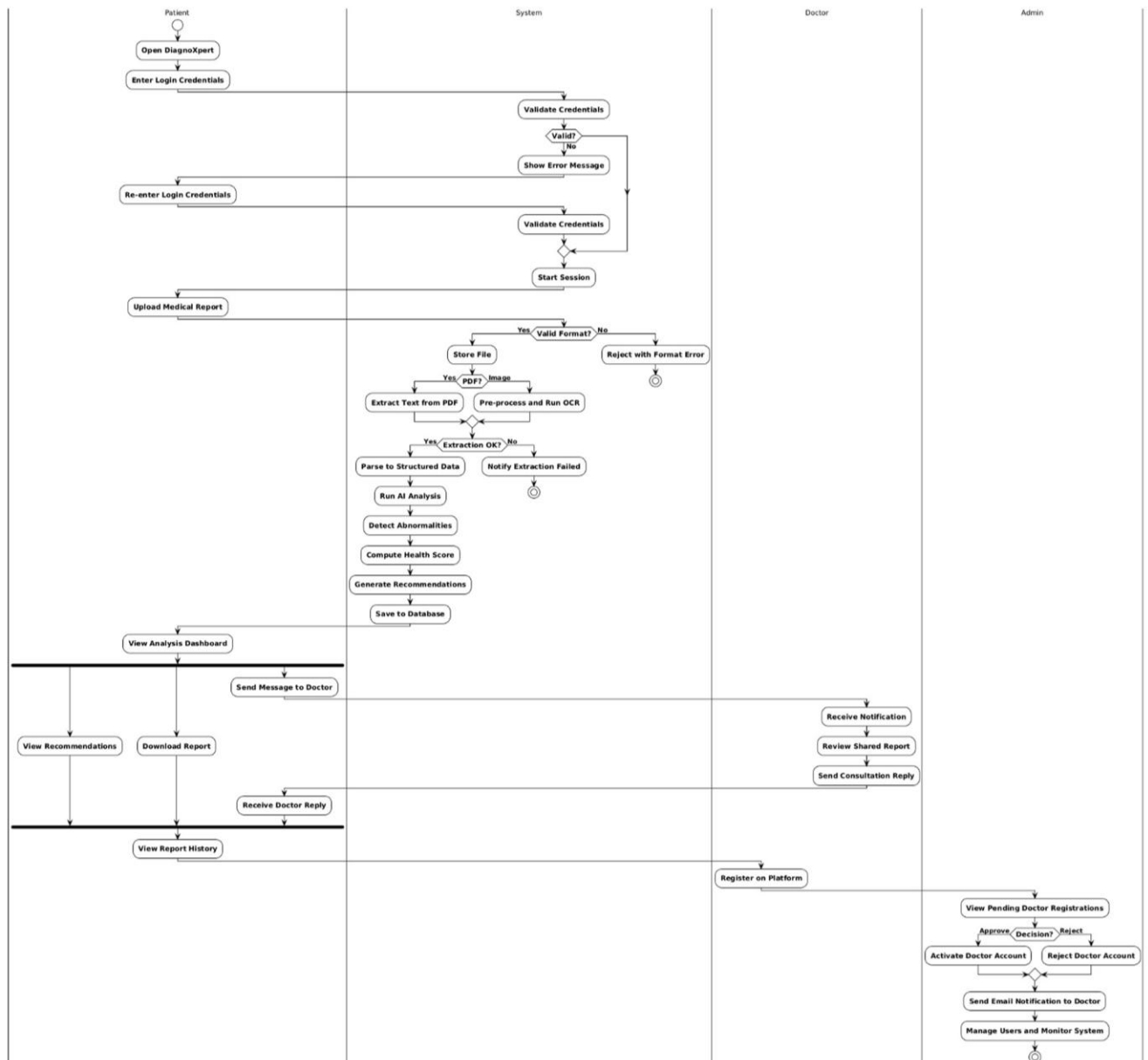


Figure 4.5 DiagnoXpert Activity Diagram

4.6 State Diagram

The state diagram below shows that how DiagnoXpert moves between states across three areas: authentication, doctor registration, and file upload/analysis.

i. Authentication

The system starts as Unauthenticated. A successful login moves it to Authenticated; wrong credentials loop it back to Unauthenticated. Too many failed attempts move it to Locked, from

which a password reset returns it to Unauthenticated. From Authenticated, logging out returns to Unauthenticated, while uploading a file moves it to Uploading.

ii. Doctor Registration

When a doctor registers, the system enters PendingApproval. An admin decision leads to either approved, after which the doctor can log in (moving to Authenticated), or rejected, which ends that path. An authenticated doctor can also submit a new registration, restarting this flow.

iii. File Upload and Analysis

After login, uploading a file enters Uploading. An invalid format moves it to InvalidFile, and re-uploading sends it back to Uploading. A valid file moves to Processing, which either succeeds (Analyzing) or fails (Failed, followed by a re-upload back to Uploading). Once analysis finishes, the system enters AnalysisDone, which runs through an internal sequence: AbnormalitiesDetected→HealthScoreReady→RecommendationsGenerated→ResultsDisplayed.

iv. After Results

From AnalysisDone, three outcomes are possible: the report is saved (Archived), downloaded (Downloaded), or sent for a doctor's consultation (ConsultationSent). In the consultation path, the doctor replies (DoctorReplied), the patient is notified (PatientNotified), and all paths converge at the final state.

Five entry points feed the system: Patient Login, Doctor Login, and Admin Login (authentication requests), Report Upload (a medical file for processing), and Chatbot Query (a health question for the AI). The three login types route to the Authentication module, Report Upload routes to File Processing, and Chatbot Query goes straight to the API Gateway.

ii. Backend Layer

The Backend contains three modules. Authentication verifies login credentials and passes the session to the API Gateway. File Processing receives uploaded reports, kicks off the OCR & AI pipeline, and stores the file in Media Storage. Consultation manages doctor–patient messaging within the backend.

iii. OCR & AI Layer

Once File Processing hands off a report, the OCR Engine extracts raw text, the Text Parser structures it into usable data, and AI Analysis generates the health findings. Results are passed on to the API Gateway.

iv. API Gateway

The API Gateway is the central hub: it receives input from Authentication, Chatbot Query, and AI Analysis, and routes data out to Notification, Admin Service, and the Database.

v. Storage Layer

The Database stores structured data (user records, analysis results, consultations), while Media Storage holds uploaded report files. Both feed the Frontend and Output layers.

vi. Frontend Layer

State Management holds the application's data, and the Dashboard UI uses it to render patient and doctor views.

vii. CI/CD Layer

Independent of user data flow: GitHub Actions runs builds and checks, and Vercel Deployment ships approved changes to production.

viii. Output Layer

The system delivers five outputs: Email Notifications, an Admin Panel, Downloadable Reports, a Doctor Dashboard, and a Patient Dashboard, drawing from the Database and Dashboard UI.

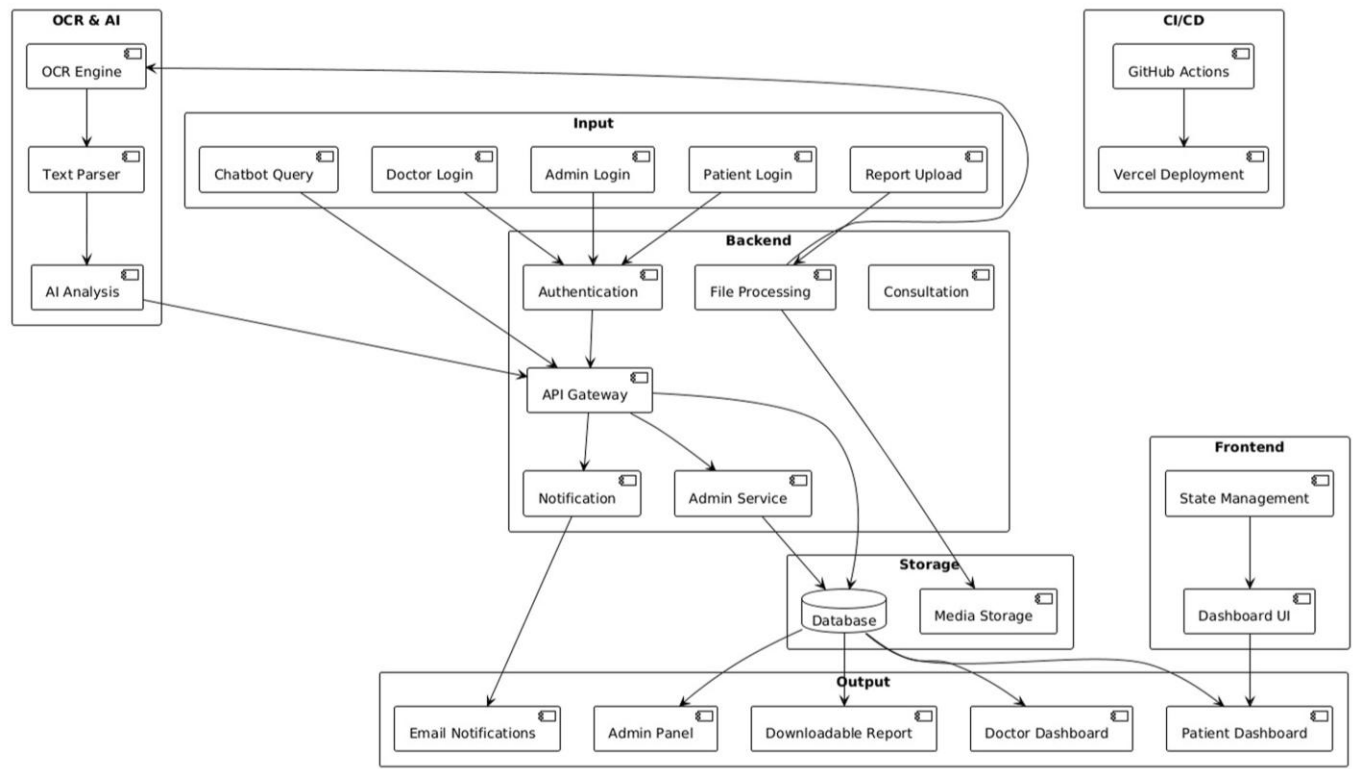


Figure 4.7 DiagnoXpert Design Flow Diagram

4.8 Entity Relationship Diagram

This Entity Relationship diagram models DiagnoXpert's MongoDB schema across seven collections. The USER collection is the central entity, storing authentication details, role (patient, doctor, or admin), and two-factor settings. A doctor-role user has a one-to-one profile in the DOCTOR collection, which holds speciality, experience, fees, credentials, and a profileStatus field (incomplete, pending, verified, or rejected) along with a rejectedFields list used during admin review.

Doctors receive patient feedback through the REVIEW collection, linked to both the doctor and the reviewing user. Messaging between users is modeled through CONVERSATION, which tracks participants and unread counts, and MESSAGE, which contains the actual text, attachments, and read status within a conversation. The MODELCHATHISTORY collection stores each user's AI chatbot interactions as an embedded array of messages. Finally, NOTIFICATION captures system alerts sent between users, distinguishing the recipient (userId) from the sender (fromUserId).

Overall, the schema separates authentication/identity (USER) from role-specific data (DOCTOR), while supporting consultation, messaging, and notification features needed for patient-doctor interaction.

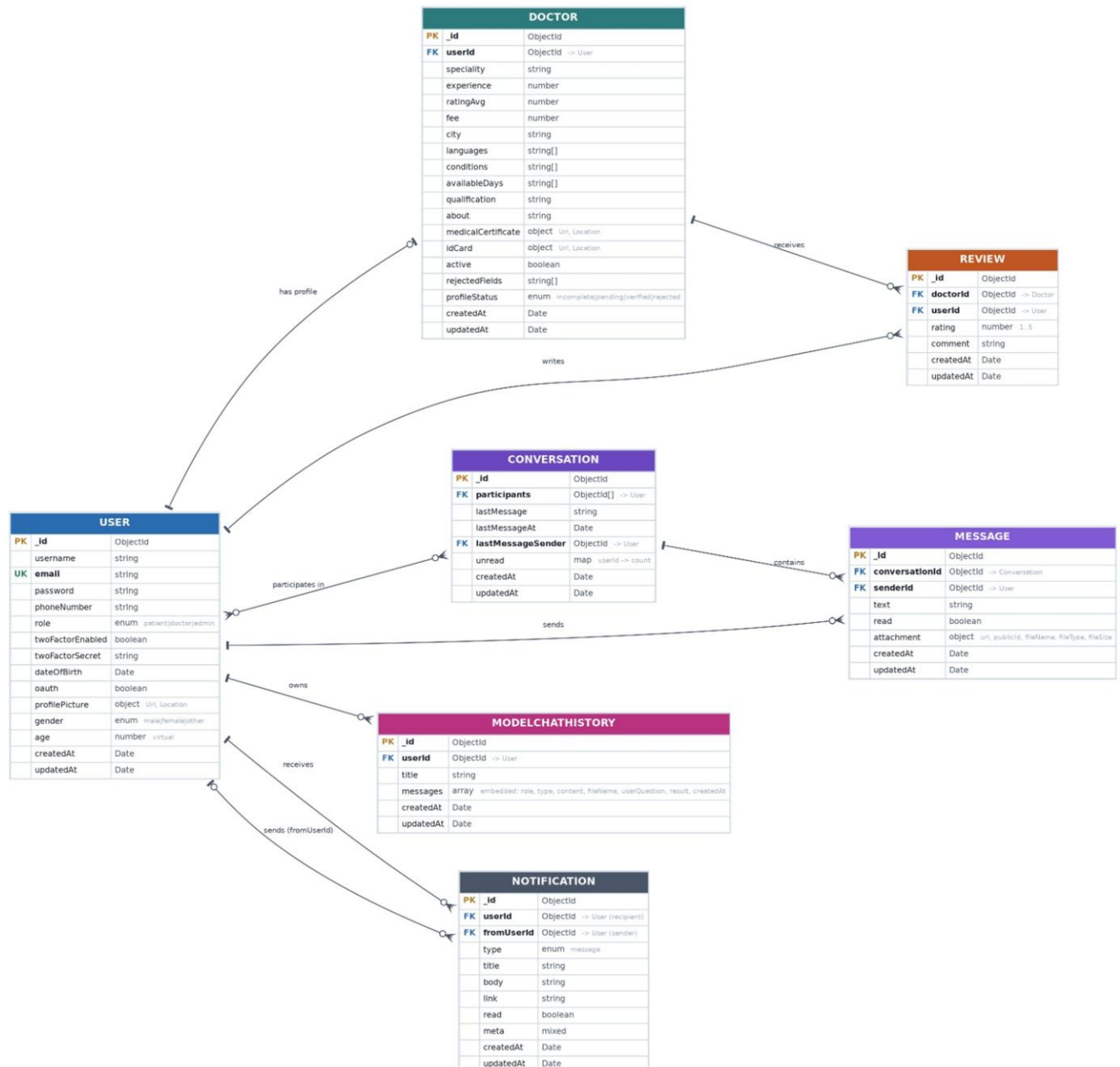


Figure 4.8 Entity Relationship Diagram

Chapter 5

Implementation

5 Implementation

In the implementation phase of DiagnoXpert, the primary goal was to create a seamless patient experience, a secure data handling pipeline, and an efficient AI driven analysis workflow. The implementation spans several interconnected subsystems that operate in coordination to deliver end to end functionality from report upload through AI generated insight delivery.

5.1 Setup Development Environment:

The DiagnoXpert development environment was established using Visual Studio Code as the primary IDE, supported by a suite of language specific extensions for Python, JavaScript, and React Native development. Git and GitHub provided the version control and team collaboration infrastructure, with a branching strategy that maintained stable main, development, and feature branches throughout the project lifecycle.

Core Tools & Configuration

i. Next.js 16 with React 19 and TypeScript for Full-Stack Development

Purpose: Serve as the unified full-stack framework handling both the frontend user interface and the backend API layer within a single codebase, eliminating the need for a separate server framework.

Setup:

- Initialized the project using the Next.js CLI with TypeScript and Tailwind CSS v4 configuration.
- Enabled Turbopack as the development bundler for significantly faster hot module replacement and build times.
- Configured the App Router with role based route protection, ensuring patients, doctors, and admins are redirected to their respective dashboards after authentication.
- Installed and configured Redux Toolkit with react redux for global state management across the application.
- Integrated Axios as the HTTP client for all internal API route communication and external Hugging Face API calls.

Key Features:

- Component based architecture built with React 19, using reusable UI elements including health score cards, report upload widgets, abnormality badges, and chatbot interfaces.
- Role based conditional rendering enforced through NextAuth.js session data patients see the report upload dashboard and analysis results, doctors see the consultation inbox and

shared reports, and admins see the admin panel for reviewing and approving or rejecting doctor registrations.

- Server side rendering and static generation used where appropriate to optimize performance and initial page load times.
- Tailwind CSS v4 and shadcn/ui used for all styling and UI components, with Lucide React and React Icons providing iconography throughout the interface.

ii. Next.js API Routes for Backend Logic

Purpose: Handle all business logic, authentication, authorization, file upload orchestration, AI pipeline triggering, consultation management, and email notification delivery entirely within the Next.js application without a separate backend server.

Setup:

- Structured API routes following a modular pattern with dedicated route handlers for authentication, report upload, OCR processing, AI analysis, consultations, notifications, and report history.
- Configured NextAuth.js with credential based authentication, role assignment (patient, doctor, admin), and secure session management using encrypted session tokens.
- Implemented bcryptjs with 12 salt rounds for password hashing prior to storage in MongoDB.
- Configured Multer middleware for multipart form data handling of medical report file uploads, with file type and size validation enforced before processing.
- Integrated Nodemailer for transactional email delivery, sending notifications for report analysis completion, consultation requests, and doctor replies.

Key Packages:

- **Nextauth:** session based authentication and role management
- **Bcryptjs:** password hashing with salt rounds
- **Multer:** File upload middleware for report processing
- **Nodemailer:** email notification service
- **Axios:** HTTP client for Hugging Face API communication
- **Mongoose:** MongoDB object modelling for schema definition and query execution

iii. OCR and Document Extraction Pipeline

Purpose: Extract raw text from uploaded medical reports regardless of whether they are digitally generated or scanned, providing clean structured input for the AI analysis pipeline.

Setup:

- Integrated pdf-parse and pdfjs-dist for direct text extraction from digitally generated, text based PDF reports.
- Integrated Tesseract.js with the English trained data file for OCR processing of scanned or image-based PDF reports.
- Integrated Sharp for image preprocessing prior to OCR operations include grayscale conversion, contrast enhancement, and noise reduction to improve Tesseract.js accuracy on low quality scans.
- Implemented a routing function that automatically determines the appropriate extraction method based on the uploaded file type.

Key Packages:

- **tesseract.js**: OCR engine for scanned PDF processing
- **pdf-parse**: text extraction from digital PDFs
- **pdfjs-dist**: PDF rendering and text layer extraction
- **sharp**: image preprocessing for OCR quality improvement

iv. Hugging Face API Integration for AI Analysis

Purpose: Perform biomedical language model inference to generate plain language report summaries, detect abnormalities, compute health scores, and produce personalized recommendations.

Setup:

- Integrated the @huggingface/inference SDK to communicate with the Llama3-OpenBioLLM model hosted on the Hugging Face Inference API.
- Structured prompts are constructed from the parsed biomarker data and submitted to the model via authenticated API calls using Axios.
- Responses are parsed and mapped to the analysis schema abnormalities list, health score, summary text, and recommendations list before being saved to MongoDB.
- Error handling and retry logic implemented for API timeout scenarios to ensure pipeline reliability.

Key Package:

- **@huggingface/inference**: SDK for authenticated Hugging Face model inference

v. MongoDB with Mongoose for Database Management

Purpose: Provide a scalable NoSQL database for all persistent data storage including user accounts, uploaded report metadata, parsed biomarker data, AI analysis results, consultation records, and notification logs.

Setup:

- Connected the Next.js application to a MongoDB instance using Mongoose ODM, with connection pooling configured for production efficiency.
- Designed document schemas for five primary collections: users, reports, analyses, consultations, and notifications.
- Implemented Mongoose schema validation, default values, and timestamps across all collections to ensure data integrity.
- Configured indexed fields on frequently queried properties such as `userId` and `reportId` to optimize query performance.

vi. Cloudinary for Media Storage

Purpose: Provide secure, scalable cloud storage for all uploaded medical report files and associated media, independent of the application server.

Setup:

- Integrated Cloudinary using the `next-cloudinary` package, configured with environment based API credentials.
- All files validated and processed by Multer are uploaded to Cloudinary, and the returned secure URL is stored in MongoDB via Mongoose for future retrieval.
- Cloudinary's built-in CDN ensures fast and reliable file access regardless of user location.

Responsive Design Implementation

DiagnoXpert's web interface is designed to render correctly across desktops, tablets, and mobile browsers. The following strategies were employed to ensure cross device compatibility.

Tailwind CSS v4's responsive utility classes were used throughout the interface to implement fluid, breakpoint driven layouts that adapt to different screen sizes without custom media query code. The `shadcn/ui` component library provides accessible, consistently styled UI components that behave correctly across all modern browsers. The application was tested on Chrome, Firefox, Edge across desktop and mobile viewports to ensure consistent behaviour and appearance. Focus areas during testing included form interactions, file upload handling, dashboard rendering at narrow viewports, and notification display behaviour.

5.2 Algorithms and External API Integration

i. Document Type Detection Algorithm

This algorithm determines how an uploaded file should be processed. When a patient uploads a PDF, the system automatically inspects the file to decide whether it contains extractable text or consists of scanned image pages. Based on this decision, the file is routed either to the PDF text extraction pipeline using pdf-parse and pdfjs-dist, or to the OCR pipeline using Tesseract.js with Sharp preprocessing. This ensures the correct extraction method is applied without requiring any manual selection by the patient.

ii. Image Preprocessing Algorithm Before Tesseract.js processes a scanned PDF page, this algorithm uses Sharp to enhance the image quality and maximize OCR accuracy. It converts the image to greyscale, normalizes contrast levels, applies sharpening to improve text edge definition, and resizes the image to an optimal resolution for OCR processing. This is particularly important for low-quality or inconsistently scanned laboratory reports.

iii. Biomarker Parsing Algorithm Once raw text is extracted from the report, this algorithm identifies and structures individual medical test parameters from the unstructured text. It scans each line of the extracted text, detects biomarker names, numerical values, measurement units, and reference ranges, and organizes them into a structured JSON format ready for analysis.

iv. Abnormality Detection Algorithm This algorithm compares each parsed biomarker value against its standard clinical reference range and assigns a severity classification Normal, Low, Moderate, or High. Each abnormal result is color coded accordingly green for normal, yellow for low severity, orange for moderate, and red for high allowing patients to immediately identify areas of concern on their dashboard.

v. Health Score Computation Algorithm This algorithm converts the complete set of biomarker results into a single numerical health score between 0 and 100, giving the patient a simple and meaningful indicator of their overall health status. Points are deducted based on the number and severity of abnormal biomarkers, with critical parameters such as haemoglobin, creatinine, and blood glucose carrying higher weighting. The final score is classified as Good, Fair, Poor, or Critical.

vi. LLM Prompt Construction Algorithm Before submitting data to the Hugging Face API, this algorithm constructs a structured, context-rich prompt from the parsed biomarker data,

abnormality flags, and health score. The prompt instructs the Llama3-OpenBioLLM model to generate a plain language report summary, per abnormality explanations, diet recommendations, lifestyle recommendations, and risk indicators with appropriate disclaimers all formatted for a non-medical patient audience.

vii. Report History and Trend Analysis Algorithm This algorithm enables longitudinal health monitoring by retrieving all past analyses for a patient from MongoDB and computing trend directions for each biomarker across multiple reports over time. It identifies whether each biomarker value is improving, stable, or worsening between reports and passes the time-series data to the frontend for graphical visualization on the patient dashboard.

5.2 External APIs

The following external APIs and services are integrated into DiagnoXpert to support AI analysis, file storage, email communication, and automated deployment. The table below summarizes each integration, followed by a brief description of its role in the system.

Table 5.1 External APIs

API / Service	Provider	Integration Method	Purpose in DiagnoXpert
Hugging Face Inference API	Hugging Face	@huggingface/inference npm SDK	LLM based medical report analysis and AI chatbot responses using Llama3-OpenBioLLM
Cloudinary API	Cloudinary	next cloudinary npm package	Secure cloud storage and CDN delivery of all uploaded medical report files
Nodemailer SMTP	SMTP Provider	nodemailer npm package	Email notifications for report completion, consultation, and email reset.
GitHub Actions	GitHub	GitHub Actions YAML workflows	Automated CI/CD pipeline for build verification and production deployment to Vercel
Vercel Deployment API	Vercel	Vercel CLI via GitHub Actions	Automated production deployment triggered on every merge to the main branch

5.3 User interface

The DiagnoXpert user interface is built with Next.js 16, React 19, Tailwind CSS v4, and shadcn/ui, designed to be clean, accessible, and easy to use for patients and doctors of all technical backgrounds. The following figures present the key screens of the platform.

5.1 Login/Signup Page

The Login/Signup page allows new users to create a DiagnoXpert account by entering their username, email address, password, date of birth, and phone number. A Sign Up button submits the form, and an option to continue with Google is also provided for convenience. A Sign In link is available for users who already have an account.

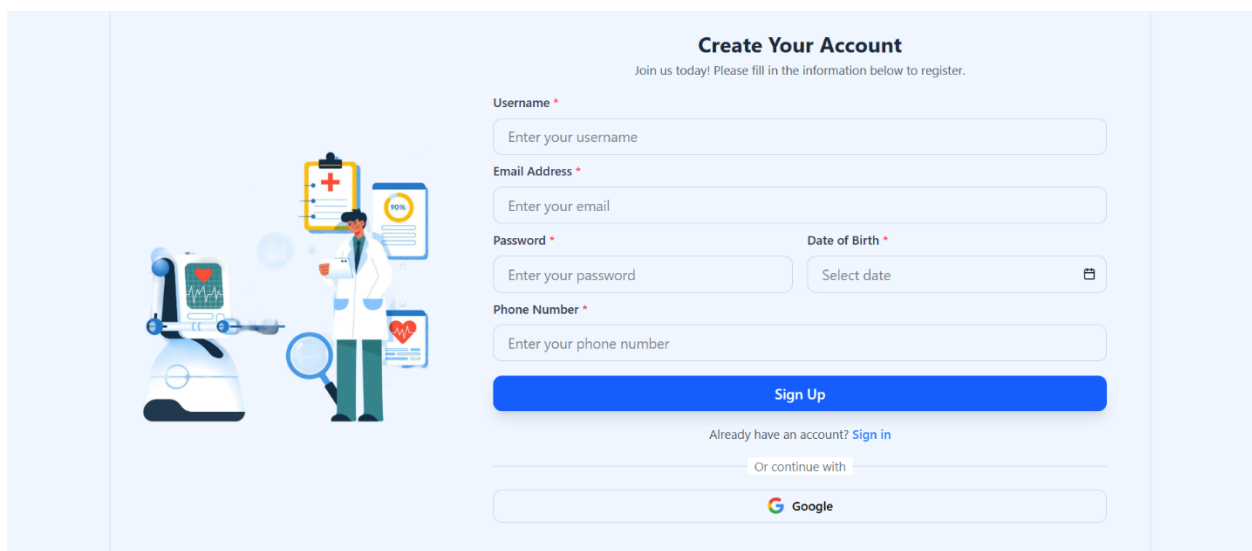
The image shows a 'Create Your Account' form on a light blue background. On the left, there is a medical illustration of a doctor in a white coat holding a magnifying glass over a patient's head, with a heart rate monitor and a clipboard nearby. The form on the right has the title 'Create Your Account' and a subtitle 'Join us today! Please fill in the information below to register.' It contains five input fields: 'Username' with a red asterisk, 'Email Address' with a red asterisk, 'Password' with a red asterisk, 'Date of Birth' with a red asterisk and a calendar icon, and 'Phone Number' with a red asterisk. Below these fields is a large blue 'Sign Up' button. Under the button, there is a link 'Already have an account? Sign in' and a section 'Or continue with' followed by a 'Google' button with the Google logo.

Figure 5.1 DiagnoXpert Login/Signup Page

5.2 Landing Page

The landing page is the first screen users see when they visit DiagnoXpert. It features a clean hero section with the platform's core value proposition AI Powered Medical Insight and Expert Consultation accompanied by Get Started and Learn More call to action buttons. The navigation bar provides quick access to Login and Sign Up, along with a dark mode toggle.

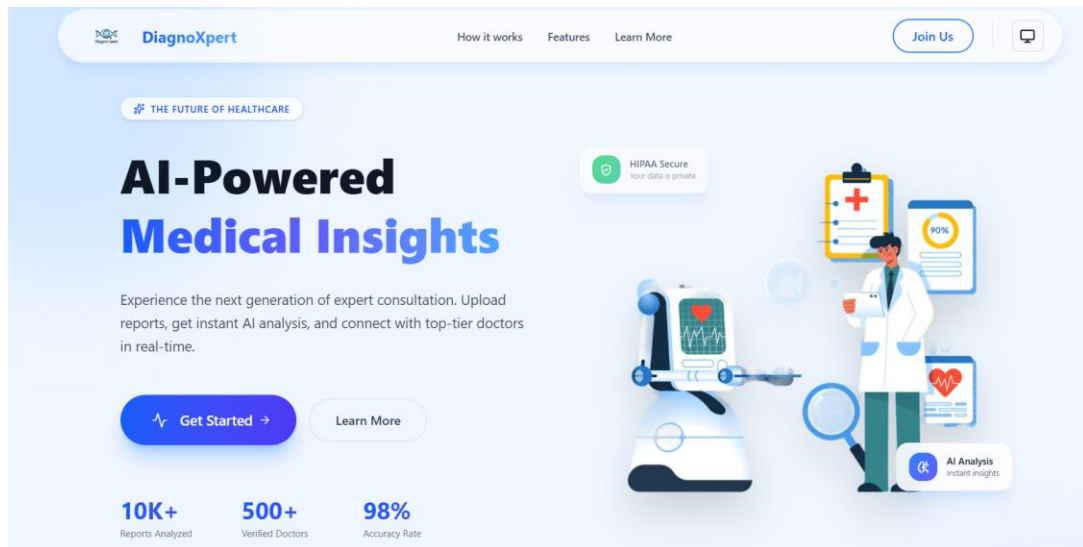


Figure 5.2 DiagnoXpert Landing Page

5.3 Feature Section

This section of the homepage explains how DiagnoXpert works in three simple steps Upload Your Reports, Analyze Report, and Chat with AI presented as clearly labelled cards with illustrative icons. Below this, a More Features row highlights the platform's key strengths: AI Insights, Secure and Private data handling, Expert Doctors, and Accessibility. This section gives new visitors a clear and immediate understanding of the platform's capabilities before they sign up.

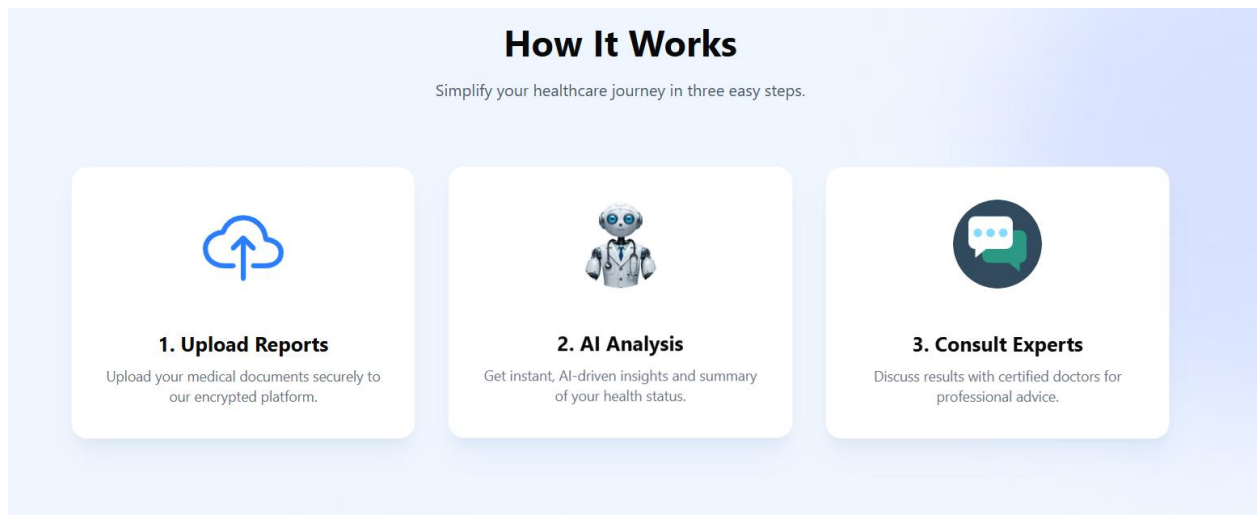
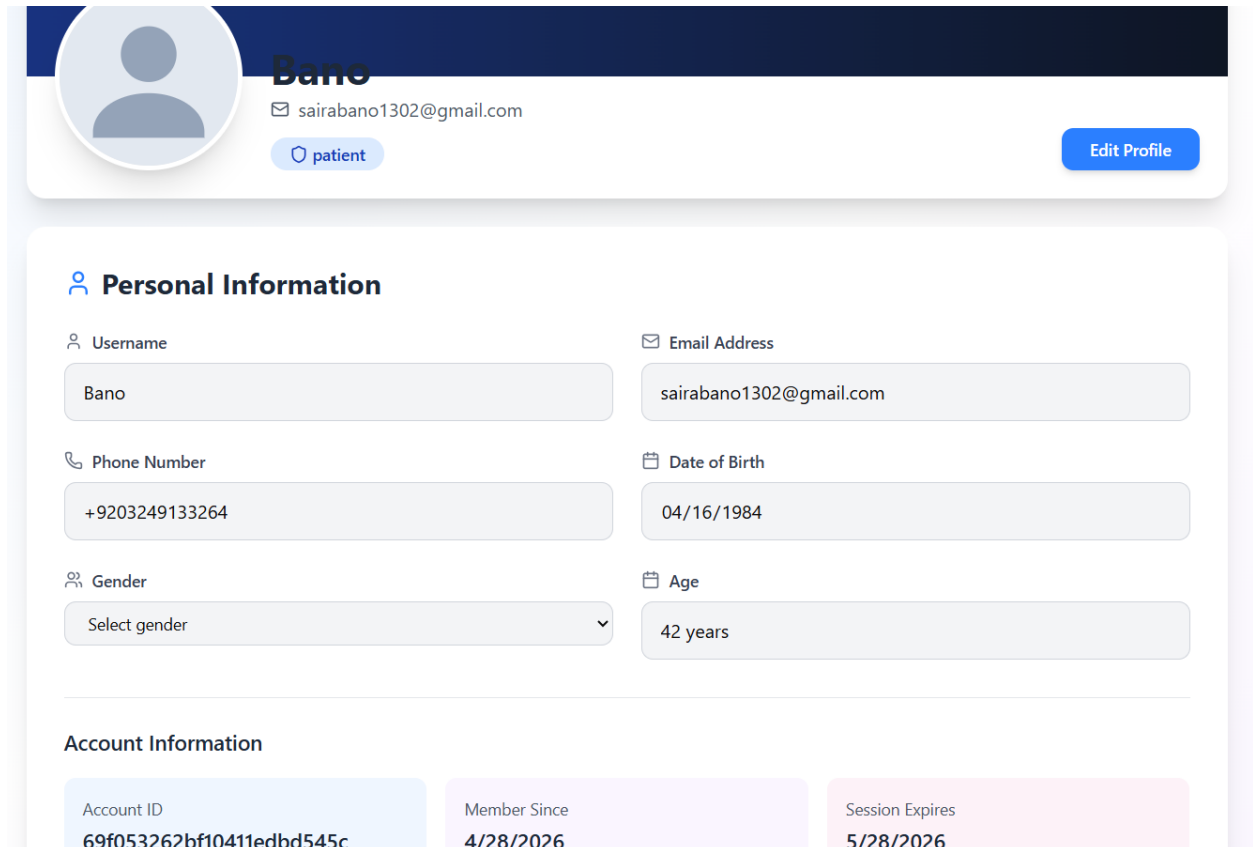


Figure 5.3 DiagnoXpert Feature Section

5.4 Patient Profile Screen

This screen displays the patient's personal profile, where they can view and update their account information including username, email address, phone number, date of birth, gender, and age. The role badge confirms the user's assigned role as patient. The Account Information section at the bottom shows the unique MongoDB generated Account ID, Member Since date, and Session



The Patient Profile Screen for Bano displays personal and account information. The header shows the user's name, email, and role (patient), with an Edit Profile button. The Personal Information section contains fields for Username, Email Address, Phone Number, Date of Birth, Gender, and Age. The Account Information section shows the Account ID, Member Since date, and Session Expires date.

Personal Information		
Username	Bano	
Email Address	sairabano1302@gmail.com	
Phone Number	+9203249133264	
Date of Birth	04/16/1984	
Gender	Select gender	
Age	42 years	

Account Information		
Account ID	Member Since	Session Expires
69f053262bf10411edbd545c	4/28/2026	5/28/2026

Figure 5.4 DiagnoXpert Patient Profile

5.5 LLM Interface

This screen is the primary report submission interface where patients upload their medical reports for AI powered analysis. The left sidebar displays the user's previous analysis history and provides access to Profile and Sign Out options. The main area welcomes the user back and presents an Attach File input combined with a Send button, allowing the patient to upload their diagnostic report either a digitally generated or scanned PDF directly for processing. Once the file is submitted, it is handled by Multer, stored in Cloudinary, and passed through the document extraction pipeline using Tesseract.js or pdf-parse depending on the file type. The extracted data is then analyzed by the Llama3-OpenBioLLM model via the Hugging Face Inference API to generate plain language health insights, abnormality detection, a personalized health score, and diet and lifestyle recommendations.

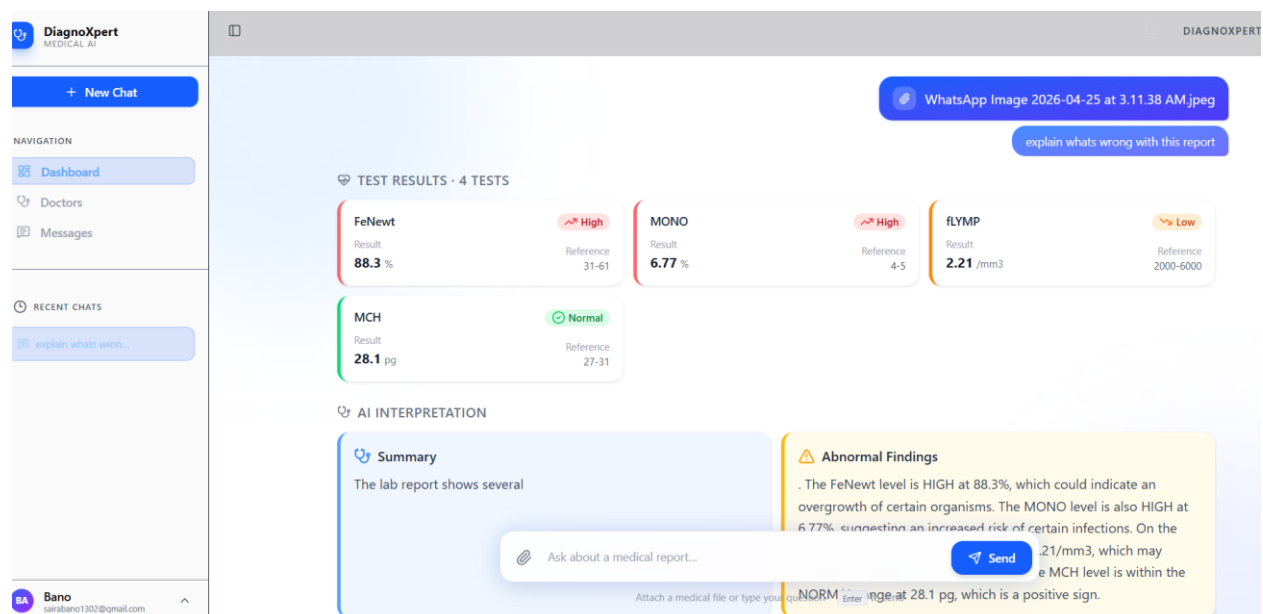


Figure 5.5 DiagnoXpert LLM Interface

5.6 Report Analysis Results Screen

This screen displays the complete results after a patient's medical report has been processed through both the OCR extraction pipeline and the Llama3-OpenBioLLM analysis engine. The top section shows Patient Info alongside an OCR Confidence score.

In this example 87.59% indicating the accuracy of text extraction performed by Tesseract.js. Below this, a structured table presents the analyzed biomarker data including Test Name, Result, Reference Value, and Unit for parameters such as Haemoglobin, Total RBC, HCT, MCV, MCH, MCHC, Platelet Count, WBC Count, and Neutrophils, with each value compared against standard

clinical reference ranges. A Change File option at the bottom allows the patient to re-upload a different report if needed.

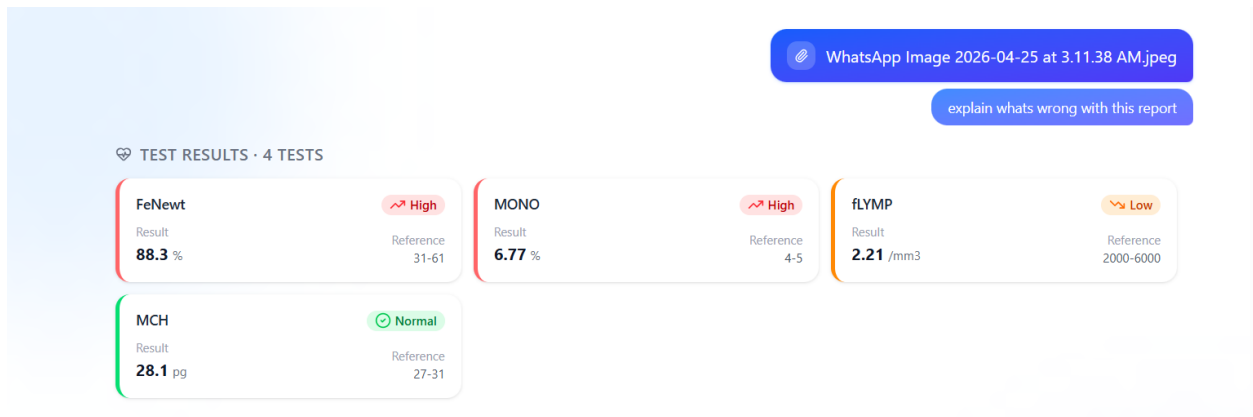


Figure 5.6 DiagnoXpert LLM Report Analysis Results

5.7 Doctor Consultation Module

This screen presents the Doctor Consultation Module, where patients can browse and connect with verified specialist doctors directly through the platform. Doctors are organized by specialization including Child Specialist, Dentist, Eye Specialist, and Gastroenterologist each displayed as a card with a relevant icon, brief description, and a Book Appointment button. Patients can select the appropriate specialist based on their report findings and initiate a consultation request, after which the doctor receives an email notification via Nodemailer and can review the shared report and AI generated analysis from their dedicated dashboard.

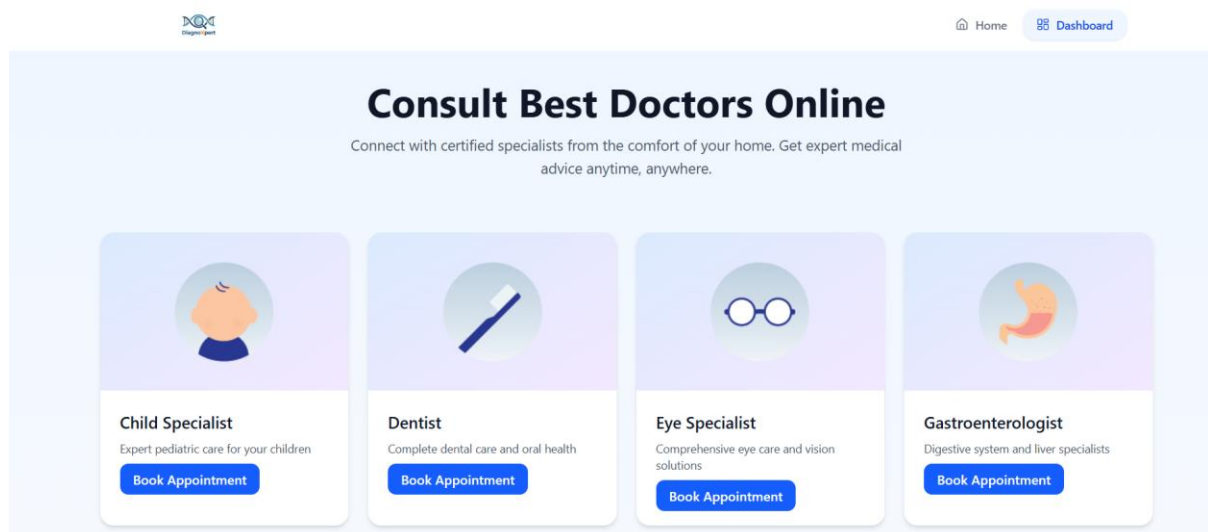


Figure 5.7 DiagnoXpert Doctor Module

5.8 Doctor Listing and Filter Screen

This screen displays the list of available doctors under a selected specialization in this case Child Specialist showing the total number of doctors currently available. Patients can filter results by City and Gender to find the most suitable doctor for their needs. Each doctor card displays the doctor's name, qualification, location, experience, online status, rating, and consultation fee. A Book Now button allows the patient to proceed directly with booking a consultation. Doctor profiles and availability data are stored and retrieved from MongoDB via Mongoose.

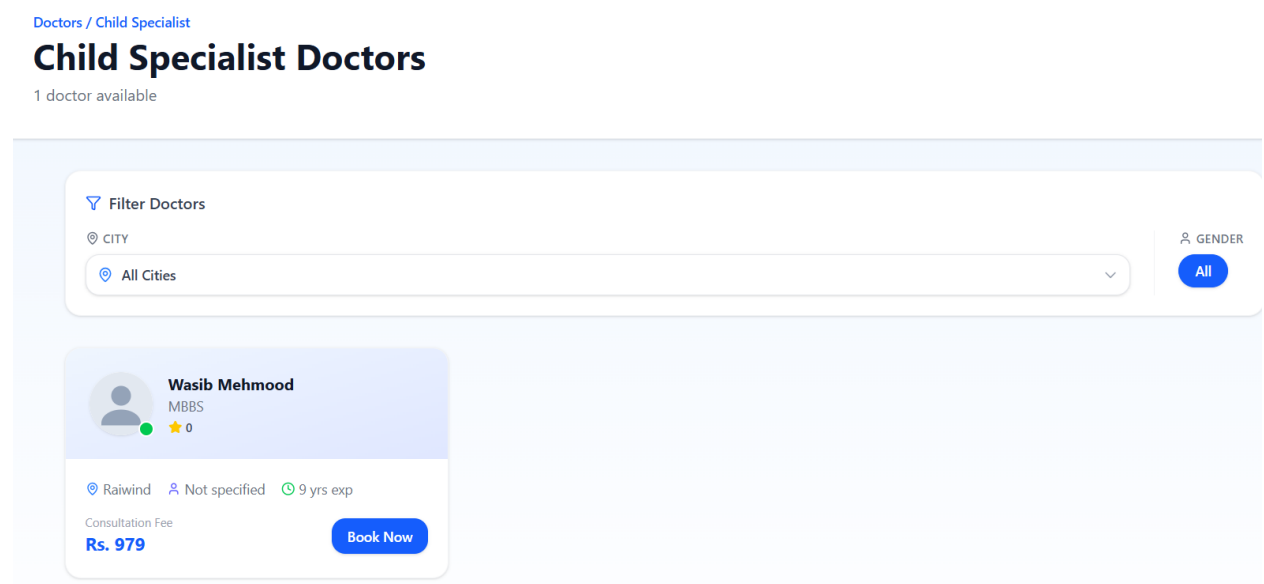


Figure 5.8 DiagnoXpert Doctor Listing and Filtering Screen

5.9 Doctor Profile Screen

This screen displays the full profile of a selected doctor, giving patients all the information they need to make an informed consultation decision. The profile includes the doctor's name, specialization, qualification, experience, location, availability status, rating, about section, and conditions treated. The right panel shows the consultation fee, contact details, languages spoken, and available days for appointments. Patients can either send a direct Message or Book Appointment from this screen, after which a consultation request is stored in MongoDB and the doctor receives an email notification via Nodemailer.

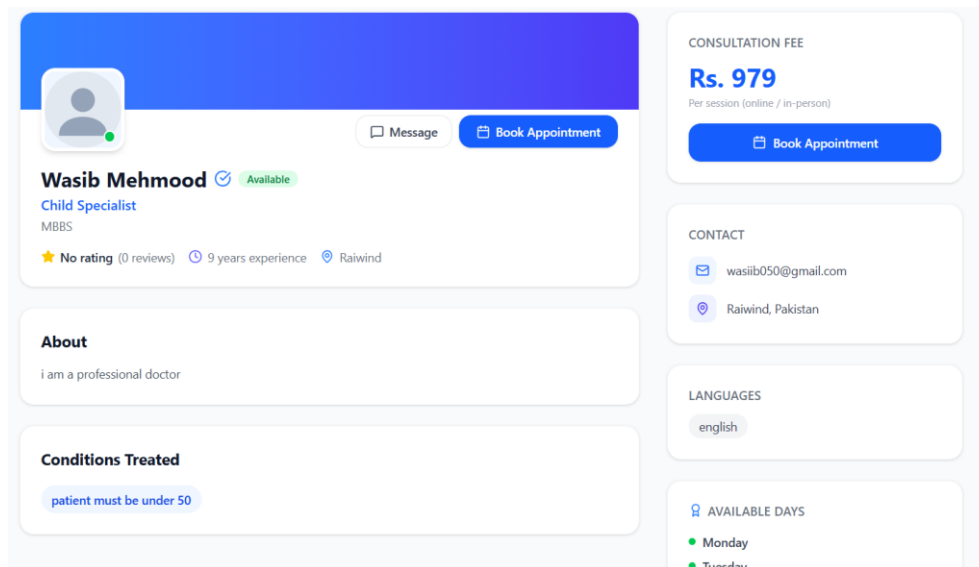


Figure 5.9 DiagnoXpert Doctor profile

5.10 Patient-Doctor Messaging Screen

This screen provides the secure messaging interface where patients can communicate directly with their selected doctor. The left sidebar displays all active conversation threads with a search bar for quick access, while the main area shows the selected conversation with the doctor's name and role displayed at the top.

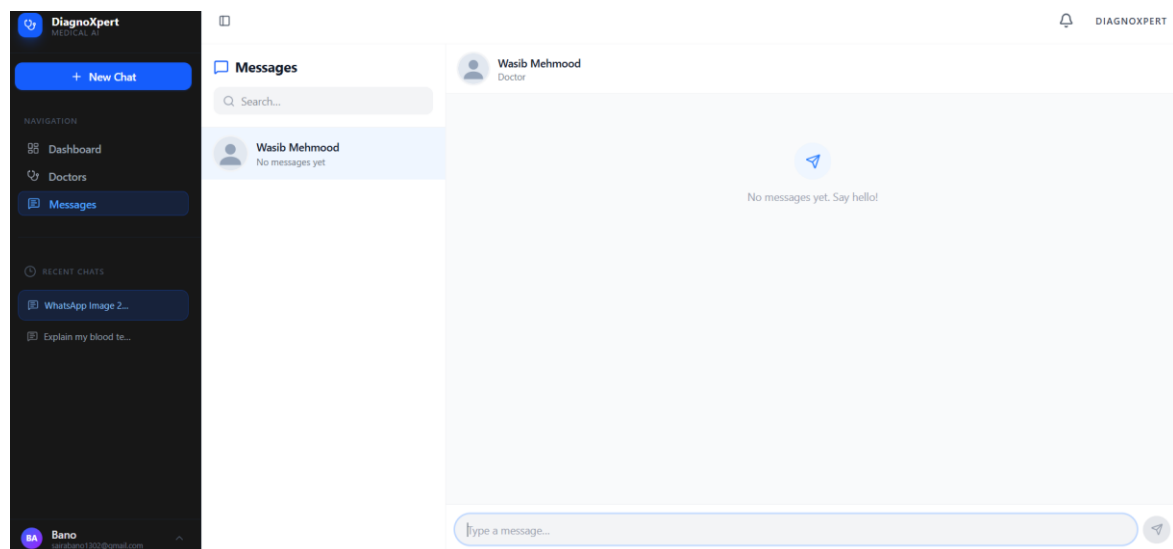


Figure 5.10 DiagnoXpert Patient-Doctor Screen

5.11 AI Chatbot Interface

This screen shows the AI powered chatbot where patients can ask health related questions in natural language by typing in the input field or attaching a medical file for context. In this example, the patient asked "Explain my blood test results" and received a context-aware response from the Llama3-OpenBioLLM model via the Hugging Face Inference API, advising the patient to consult a doctor for specific report interpretation. The clean, minimal interface with an "Ask about a medical report..." placeholder guides patients to interact naturally, and messages can be sent by clicking the Send button or pressing Enter.

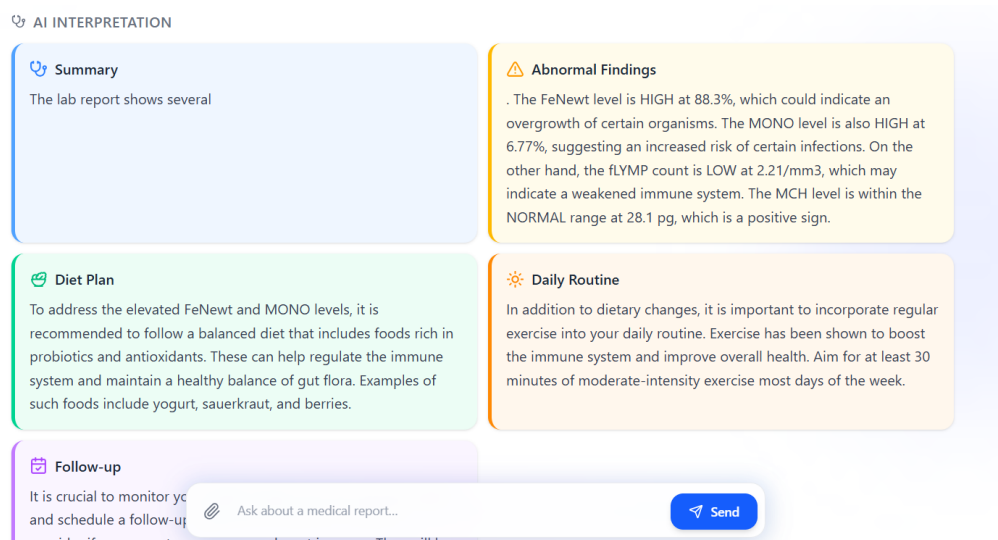


Figure 5.11 DiagnoXpert AI Chatbot Interface

5.12 Admin Login Screen

The admin login interface requires an email and password before issuing a one-time password, adding an extra verification layer beyond standard credential checks. This two-step design protects the admin panel, which controls doctor approval and platform-wide oversight.

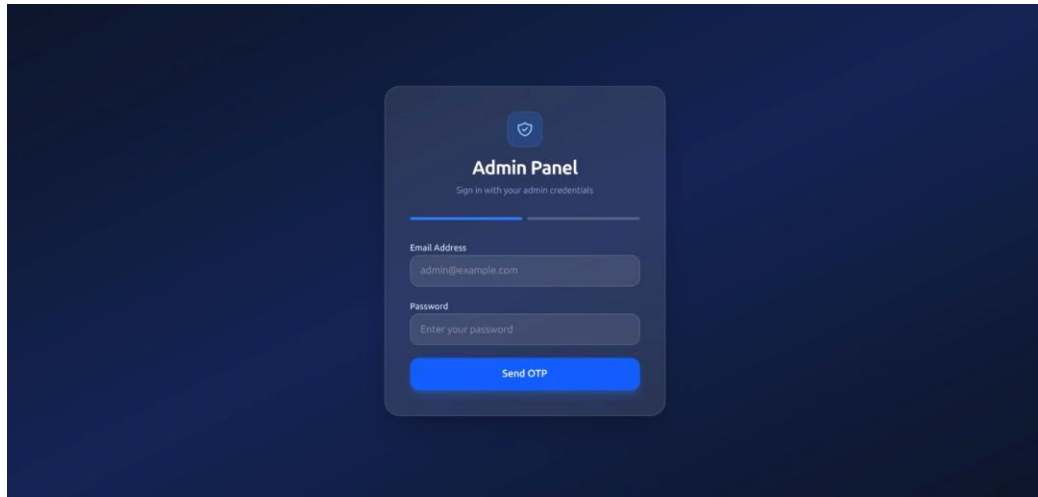


Figure 5.12 DiagnoXpert Admin Login Screen

5.13 Admin OTP Verification

After submitting valid credentials, the admin receives a one-time password via email, shown here with a 10-minute validity window. This step confirms the admin's identity before granting access to the dashboard.

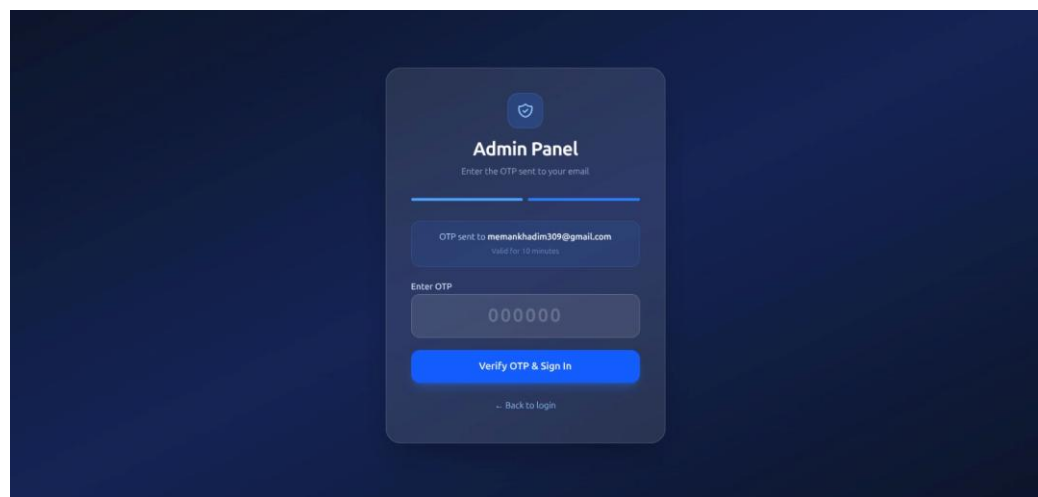


Figure 5.13 DiagnoXpert Admin OTP Verification

5.14 Admin Dashboard Overview

The admin dashboard presents a real-time summary of platform activity, including total users, patients, and doctors, along with counts of incomplete, pending, verified, and rejected doctor profiles. This gives the admin an at-a-glance view of registration activity requiring attention.

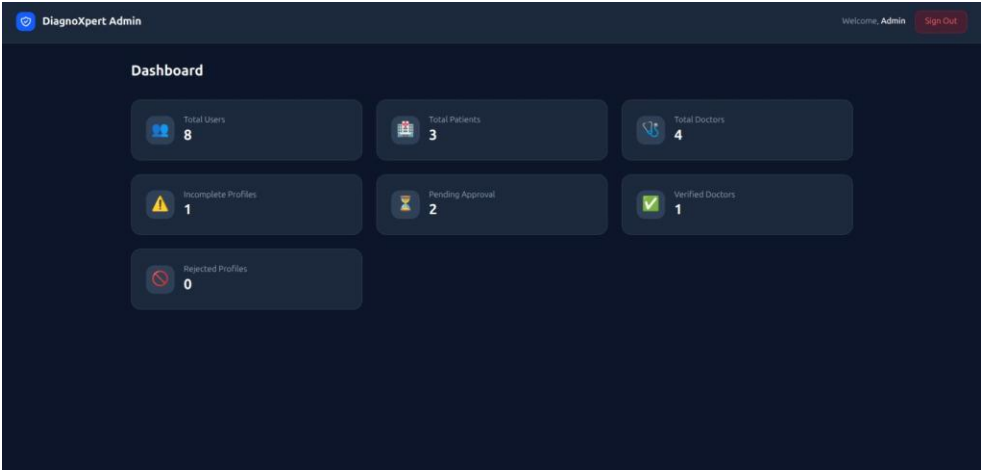


Figure 5.14 DiagnoXpert Admin Dashboard Overview

5.15 Doctor Profile Review and Field-Level Approval

This screen allows the admin to review a pending doctor's submitted details field by field, selectively accepting or rejecting individual entries such as qualification, experience, or available days. Rejected fields are flagged for the doctor to correct, while the admin can either reject the listed issues or verify the entire profile at once.

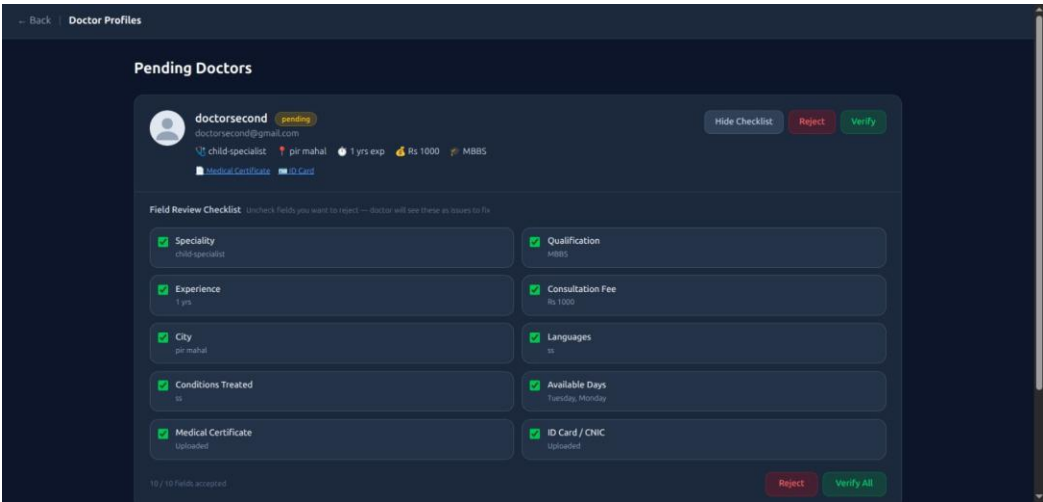


Figure 5.15 DiagnoXpert Doctor Profile Review and Field-Level Approval

Chapter 6

Testing and Evaluation

6 Testing

A comprehensive, multi layered testing strategy was adopted for DiagnoXpert, encompassing unit testing, functional testing, integration testing, system testing, and manual user acceptance testing. Given the medical nature of the application, particular emphasis was placed on accuracy validation for the AI analysis and risk scoring modules, ensuring that generated insights aligned with established clinical reference ranges. All testing was conducted against a dedicated test environment using anonymized synthetic patient data to safeguard data privacy. The CI/CD pipeline via GitHub Actions ensured that every code change was automatically built and verified on Node.js 20.x and 22.x before being merged, providing continuous regression protection throughout the development lifecycle.

6.1 System Testing

Once DiagnoXpert was successfully developed, testing was performed to ensure the system works as intended and meets all requirements stated earlier. System testing helped identify errors that may be hidden from the user. The types of testing conducted include unit testing, functional testing, and integration testing. All testing was completed before deployment to Vercel.

6.2 Unit Testing

Unit testing verifies that individual components and functions of the system work correctly in isolation.

Unit Testing 1: Patient Login

Testing Objective: To ensure the login form works correctly for patients.

Table 6.2 Unit Testing 1: Patient Login

No.	Test Script	Attribute Value and Expected Result	Result
1	Verify patient login with correct credentials	Email: patient@gmail.com / Password: correct password. Successfully logged in and redirected to patient dashboard	Pass
2	Verify patient login with incorrect credentials	Email: patient@gmail.com / Password: wrong password. Error message displayed; account locked after 5 failed attempts	Pass

3	Verify account lock after 5 failed login attempts	Incorrect password entered 5 times. Account should be locked temporarily	Fail
---	---	--	------

Unit Testing 2: Patient Profile Update

Testing Objective: To ensure the profile edit form works correctly.

Table 6.3 Unit Testing 2: Patient Profile Update

No.	Test Script	Attribute Value and Expected Result	Result
1	Update username and phone number	New username and phone entered Profile updated successfully and saved to MongoDB	Pass
2	Submit profile form with empty required fields	Required fields left blank Validation error displayed; form not submitted	Pass
3	Update profile picture with unsupported image format	Unsupported file selected. Upload should be rejected	Fail

Unit Testing 3: Report Upload

Testing Objective: To ensure the report upload function works correctly.

Table 6.4 Unit Testing 3: Report Upload

No.	Test Script	Attribute Value and Expected Result	Result
1	Upload a valid digital PDF report	PDF file selected, File processed by Multer, stored in Cloudinary, extraction triggered	Pass
2	Upload an unsupported file format	Non PDF file selected, Error message displayed with format guidance	Pass
3	Upload large PDF report (>10MB)	File should upload successfully or show size limit warning	Fail

Unit Testing 4: Admin

Testing Objective: To ensure the admin panel correctly displays and processes pending doctor registrations.

Table 6.5 Unit Testing 4: Doctor Registration Approval

No.	Test Script	Attribute Value and Expected Result	Result
1	Verify pending doctor registrations are listed	Doctor submits registration; admin opens admin panel; registration appears in the pending list	Pass
2	Approve a pending doctor registration	Admin selects a pending doctor and clicks Approve; status updates to Approved; doctor can now log in	Pass
3	Reject a pending doctor registration	Admin selects a pending doctor and clicks Reject; status updates to Rejected; doctor remains unable to log in	Pass
4	Action buttons disabled for already processed registration	Admin reopens an already approved/rejected entry; Approve/Reject buttons should be disabled or hidden	Fail

6.3 Functional Testing

Functional testing takes place after unit testing to verify that each module of DiagnoXpert meets its specified requirements.

Functional Testing 1: Login with Different Roles

Objective: To ensure that the correct dashboard is loaded based on the user's assigned role.

Table 6.6 Functional Testing 1: Login with Different Roles

No.	Test Script	Attribute and value	Expected Result	Result
1	Login as a Patient	Email: patient@gmail.com Password: correct password	Patient dashboard loaded with report upload and analysis features	Pass

2	Login as a Doctor	Email: doctor@gmail.com Password: Correct password	Doctor dashboard loaded with consultation inbox and shared reports	Pass
3	Patient accesses doctor dashboard URL directly	Access should be denied	Access denied and redirected to patient dashboard	Fail
4	Doctor accesses patient only upload page	Doctor tries to access patient upload section	Access should be restricted	Fail

Functional Testing 2: Report Analysis

Objective: To ensure the OCR extraction and AI analysis pipeline produces correct output.

Table 6.7 Functional Testing 2: Report Analysis

No.	Test Script	Attribute	Expected Result	Result
1	Upload digital PDF report	Valid CBC report in PDF format	Text extracted via pdf parse; biomarkers parsed; AI analysis generated	Pass
2	Upload scanned PDF report	Scanned lab report image	Image preprocessed by Sharp; OCR run by Tesseract.js; analysis generated	Pass
3	Upload low quality scanned report	OCR should extract text correctly or show an error message	OCR should extract readable text or show error	Fail
4	Upload unsupported report format	Unknown file format uploaded	System should reject file	Fail

Functional Testing 3: Admin Login and Access Control

Objective: To ensure the admin role loads the correct dashboard and that access is properly restricted between roles.

Table 6.8 Functional Testing 3: Admin Login and Access Control

No.	Test Script	Attribute and Value	Expected Result	Result
1	Login as Admin	Email: admin@gmail.com / Password: correct password	Admin panel loaded with list of pending doctor registrations	Pass
2	Patient accesses admin panel URL directly	Patient session active, navigates to /admin	Access denied and redirected to patient dashboard	Fail
3	Doctor accesses admin panel URL directly	Doctor session active, navigates to /admin	Access denied and redirected to doctor dashboard	Fail
4	Admin accesses patient/doctor only pages	Admin tries to access patient upload page	Access should be restricted	Fail

6.4 Integration Testing

Integration testing verifies that all modules of DiagnoXpert work correctly together as a complete system.

Table 6.9 Integration Testing

No.	Test Script	Attribute and value	Expected Result	Result
1	Patient registers and logs in	Email and password entered on the registration form	Account created in MongoDB; NextAuth.js session started; dashboard displayed	Pass

2	Upload report and the view analysis	Valid PDF report uploaded	File stored in Cloudinary; text extracted; LLM analysis generated and displayed on dashboard	Pass
3	Patient contacts doctor	Doctor selected and consultation request submitted	Request stored in MongoDB; doctor receives email notification via Nodemailer	Pass
4	Doctor reviews report and the replies	Doctor opens shared report and submits reply	Reply saved to MongoDB; patient receives email notification via Nodemailer	Pass
5	View the report history	Patient navigates to report history section	All past reports and analyses retrieved from MongoDB and displayed chronologically	Pass
6	Upload report with broken session	Patient uploads report after session expiry	System should redirect to login and prevent upload	Fail

Table 6.10 Integration Testing

No.	Test Script	Attribute and Value	Expected Result	Result
7	Doctor registers and awaits approval	Doctor submits registration form	Account created with status Pending; doctor unable to log in until approved	Pass
8	Admin approves doctor registration	Admin reviews and approves the pending doctor	Status updates to Approved; doctor can now log in successfully	Pass

6.5 Automated Testing

Table 6.11 Automated Testing

Tool Name	Description	Applied On
GitHub Actions	Automated CI pipeline that runs npm ci and npm run build on every pull request to verify the codebase compiles without errors	All functional requirements build verification before every merge
Node.js 20.x and 22.x	Runtime environments used by GitHub Actions to run the automated build and test process	FR-001 to FR-014, NFR performance and compatibility requirements
Vercel CLI	Automated deployment tool triggered on merge to main branch runs vercel pull, vercel build prod, and vercel deploy prod	Full system deployment verification after every successful merge
GitHub Actions	Build pipeline intentionally fails when missing environment variables (.env)	Deployment pipeline validation

6.6 Results

All automated build checks passed consistently across Node.js 20.x and 22.x throughout the development lifecycle. No build failures were recorded upon merge to the main branch, and all production deployments to Vercel completed successfully via the automated CI/CD pipeline

Chapter 7

Conclusion and Future Work

7 Conclusion and Future Work

7.1 Conclusion

The DiagnoXpert represents a meaningful and technically substantive contribution to the domain of patient centered digital healthcare. By combining the complementary capabilities of Optical Character Recognition, Large Language Models, and machine learning within a unified and accessible web platform, the system successfully addresses the longstanding challenge of medical report interpretation for non-clinical users a challenge that is particularly acute in Pakistan and across the developing world, where access to timely physician consultation remains limited.

The platform delivers on its core objectives. It accurately extracts and parses biomarker data from diverse report formats using Tesseract.js and pdf-parse. It generates clinically grounded, patient friendly analysis through the Llama3-OpenBioLLM model accessed via the Hugging Face Inference API. It provides personalized health scoring and severity graded abnormality detection. It enables direct doctor consultation through an integrated messaging module. It offers an AI powered chatbot for interactive health query resolution. And it maintains platform integrity through an admin verification workflow that screens and approves doctor registrations before consultation access is granted. The system is built on a modern, modular, and scalable technology stack comprising Next.js 16, React 19, TypeScript, MongoDB with Mongoose, NextAuth.js, Cloudinary, and an automated CI/CD pipeline through GitHub Actions and Vercel providing a solid and maintainable foundation for future expansion.

Testing across all modules confirmed that the system meets its functional and non-functional requirements. Manual, unit, functional, and integration tests produced consistent pass results, validating the correctness and reliability of the implementation across all core workflows, including authentication, role-based access control, and the doctor approval process. The AI analysis pipeline demonstrated accurate identification of common haematological and metabolic abnormalities across a diverse set of synthetic test reports, with generated insights evaluated as clinically relevant and patient comprehensible.

DiagnoXpert does not seek to replace the physician. Rather, it seeks to empower the patient providing the knowledge, context, and confidence needed to engage more meaningfully with

the healthcare system. In doing so, it contributes to a broader vision of a more equitable, informed, and patient centered model of healthcare delivery in Pakistan and beyond.

7.2 Future Work

While DiagnoXpert has successfully achieved its initial project scope, several valuable directions for future development have been identified during the course of the project.

Radiology Report Analysis Extending the platform's analytical scope to include AI based interpretation of Xray, MRI, and CT scan images using computer vision models would transform DiagnoXpert into a significantly more comprehensive diagnostic support tool, covering both laboratory and imaging investigations.

EMR and Hospital System Integration Integrating DiagnoXpert with hospital management systems and Electronic Medical Record platforms would enable direct import of diagnostic data and two way clinical workflow integration, making the platform suitable for deployment within formal healthcare institutions.

Real-Time Health Monitoring Exploring integration with wearable health devices such as smartwatches and continuous glucose monitors would allow DiagnoXpert to provide ongoing health trend tracking and proactive alert generation based on real time physiological data.

Expanded Language Support Adding multilingual support particularly Urdu, Punjabi, and other regional Pakistani languages would significantly increase accessibility for non-English speaking patients, directly strengthening the platform's mission of democratizing healthcare information.

Clinical Validation Study Conducting a structured clinical validation study in collaboration with qualified healthcare professionals would rigorously assess the accuracy, reliability, and clinical utility of DiagnoXpert's AI generated analysis outputs, providing the empirical evidence needed to support broader adoption.

AI Model Fine Tuning the Llama3-OpenBioLLM model on Pakistan specific laboratory reference ranges, local disease prevalence patterns, and regional dietary and lifestyle factors would improve the clinical relevance and personalization of the platform's health insights for the target population.

Prescription Support Module Adding functionality for doctors to issue electronic prescriptions directly within the platform would create a more complete digital clinical encounter record, reducing the need for patients to manage separate paper-based prescriptions alongside their digital health data.

Mobile Application Developing a dedicated mobile application for iOS and Android would extend DiagnoXpert's reach to patients in areas where desktop access is limited, further supporting the platform's goal of making healthcare information accessible to everyone regardless of their location or device.

8 References

1. Kumar, A., Dwivedi, A.K., Singh, S., Bhardwaj, A., Kumar, A. and Dong-Nhat, L. (2022) 'Evolving Predictive Analytics in Healthcare: New AI Techniques for Real-Time Interventions'. London: The Institution of Engineering and Technology (IET).
2. Huang, J., Tan, J. et al. (2025) 'Enhancing Medical Diagnosis Document Analysis with Layout-Aware Vision-Language Models'. *Diagnostics*. [Online]. Available at: <https://doi.org/10.3390/diagnostics> (Accessed: 2025).
3. Johnson, A., Bretz, B. et al. (2025) 'Benefits, limits, and risks of ChatGPT in medicine'. *Frontiers in Artificial Intelligence*. [Online]. Available at: <https://www.frontiersin.org> (Accessed: 2025).
4. Johnson, B., Bretz, B. et al. (2025) 'Leveraging large language models for structured information extraction from pathology reports'. *Journal of Pathology Informatics*. [Online]. Available at: <https://doi.org/10.1016/j.jpi> (Accessed: 2025).
5. Kerr, C. et al. (2026) 'AI-driven analysis of patient safety reports using large language models: an exploratory multiple methods study'. *BMJ Quality & Safety*. [Online]. Available at: <https://qualitysafety.bmj.com> (Accessed: 2026).
6. Li, B. et al. (2025) 'Scientific evidence for clinical text summarization using large language models: Scoping review'. *Journal of Medical Internet Research*, 27. [Online]. Available at: <https://www.jmir.org> (Accessed: 2025).
7. Saikia, M. et al. (2025) 'Large language models in healthcare and medical applications'. *PMC / Bioengineering*. [Online]. Available at: <https://www.ncbi.nlm.nih.gov/pmc> (Accessed: 2025).
8. DIGI-TEXX (2025) 'The Role of OCR in Healthcare'. *DIGI-TEXX TechBlog*. [Online]. Available at: <https://digi-texx.com/techblog/the-role-of-ocr-in-healthcare/> (Accessed: 2025).
9. Eysenbach, G. (2001) 'What is e-health?'. *Journal of Medical Internet Research*, 3(2), p.e20. [Online]. Available at: <https://www.jmir.org> (Accessed: 2025).
10. Gunčar, G., Kukar, M., Notar, M., Brvar, M., Černelč, P., Notar, M. and Notar, M. (2018) 'An application of machine learning to haematological diagnosis'. *Scientific Reports*, 8(1), p.411.
11. Friedman, C. and Hripcsak, G. (1999) 'Natural language processing and its future in medicine'. *Academic Medicine*, 74(8), p.890.
12. Ponnuru, M., Patel, S., Agrawal, L., Bhati, T., Singh, S. and Chandra, G. (2024) 'Image-based extraction of prescription information using OCR-Tesseract'. *Procedia Computer Science*, 235, pp.1234–1241.
13. Berkman, N.D., Sheridan, S.L., Donahue, K.E., Halpern, D.J., Viera, A. and Crotty, K. (2011) 'Low health literacy and health outcomes: An updated systematic review'. *Annals of Internal Medicine*, 155(2), pp.97–107.

14. Kazakov, O.D., Vlasenko, A.V. and Kazakova, N.Y. (2019) 'Machine learning in the analysis of medical data'. *Journal of Physics: Conference Series*, 1210, p.012017.
15. Saikia, S.M. and Joshi, M.J. (2025) 'Large language models in healthcare and medical applications: A review'. *Bioengineering*, 12(6).
16. Saifullah, M., Ma, Y., Li, X., Maqbool, A. and Chen, Z. (2024) 'Enhancing telehealth services development in Pakistani healthcare sectors through examining various medical service quality characteristics'. *Frontiers in Public Health*, 12, p.1376534. [Online]. Available at: <https://doi.org/10.3389/fpubh.2024.1376534> (Accessed: 2025).
17. Irfan, B. and Yaqoob, A. (2024) 'The digital lifeline: telemedicine and artificial intelligence synergy as a catalyst for healthcare equity in Pakistan'. *Cureus*, 16(2), p.e54017. [Online]. Available at: <https://doi.org/10.7759/cureus.54017> (Accessed: 2025).
18. Faiyazuddin, M. et al. (2025) 'The impact of artificial intelligence on healthcare: A comprehensive review of advancements in diagnostics, treatment, and operational efficiency'. *Health Science Reports*, 8(1), p.e70312. [Online]. Available at: <https://doi.org/10.1002/hsr2.70312> (Accessed: 2025).
19. Fitzpatrick, P.J. (2023) 'Improving health literacy using the power of digital communications to achieve better health outcomes for patients and practitioners'. *Frontiers in Digital Health*, 5, p.1264780. [Online]. Available at: <https://doi.org/10.3389/fdgth.2023.1264780> (Accessed: 2025).
20. Singhal, K., Azizi, S., Tu, T., Mahdavi, S.S., Wei, J., Chung, H.W., Scales, N., Tanwani, A., Cole-Lewis, H., Pfohl, S. et al. (2023) 'Large language models encode clinical knowledge'. *Nature*, 620(7972), pp.172–180. [Online]. Available at: <https://doi.org/10.1038/s41586-023-06291-2> (Accessed: 2025).
21. Luo, R., Sun, L., Xia, Y., Qin, T., Zhang, S., Poon, H. and Liu, T.Y. (2022) 'BioGPT: generative pre-trained transformer for biomedical text generation and mining'. *Briefings in Bioinformatics*, 23(6), p.bbac409. [Online]. Available at: <https://doi.org/10.1093/bib/bbac409> (Accessed: 2025).
22. Abbas, M.W., Tahir, H.N., Jaffar, N., Raza, G., Ali, S.A. and Jabeen, R. (2024) 'Facilitators and barriers in acceptance of telemedicine among healthcare providers in Pakistan: A cross-sectional survey'. *Journal of Medical Access*, 8. [Online]. Available at: <https://doi.org/10.1177/27550834241266413> (Accessed: 2025).
23. Dey, S. and Pradhan, R. (2022) 'AI-backed OCR system for healthcare record digitization using CNN-RNN'. *Procedia Computer Science*, 199, pp.1145–1152. [Online]. Available at: <https://doi.org/10.1016/j.procs.2022.01.145> (Accessed: 2025).
24. Paul, M., Maglaras, L., Ferrag, M.A. and Almomani, I. (2023) 'Digitization of healthcare sector: a study on privacy and security concerns'. *ICT Express*, 9(4), pp.571–588. [Online]. Available at: <https://doi.org/10.1016/j.icte.2023.02.007> (Accessed: 2025).